

Data Communications

Lecture #10

What we have discussed

- Connection-oriented transport: TCP

Today...

- Principles of congestion control
- TCP congestion control

Principles of congestion control

Principles of congestion control

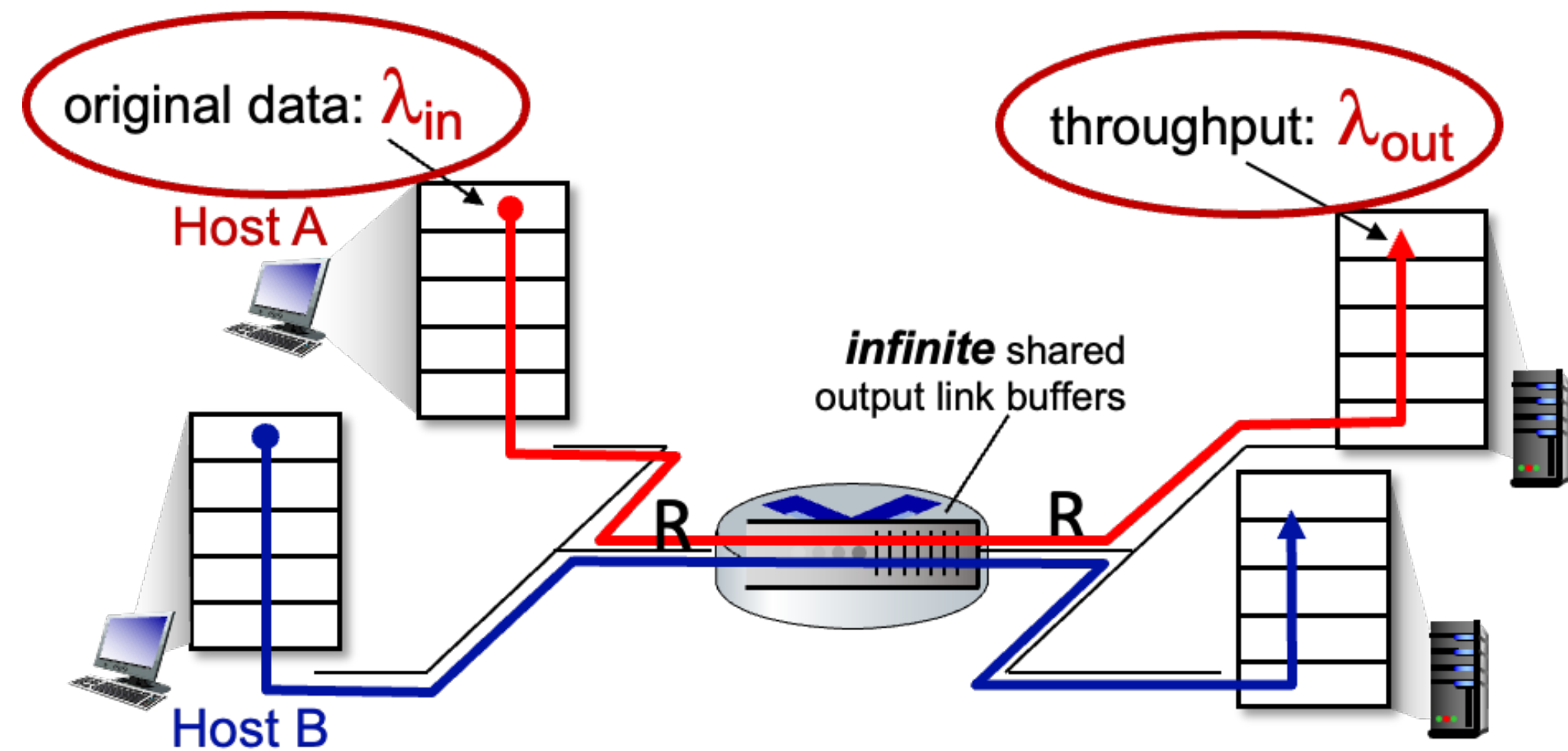
- Congestion
 - Informally: “too many sources sending too much data too fast for network to handle”
 - Manifestations
 - Long delay (queueing in router buffers)
 - Packet loss (buffer overflow at routers)
 - Different from flow control!
 - A top-10 problem!



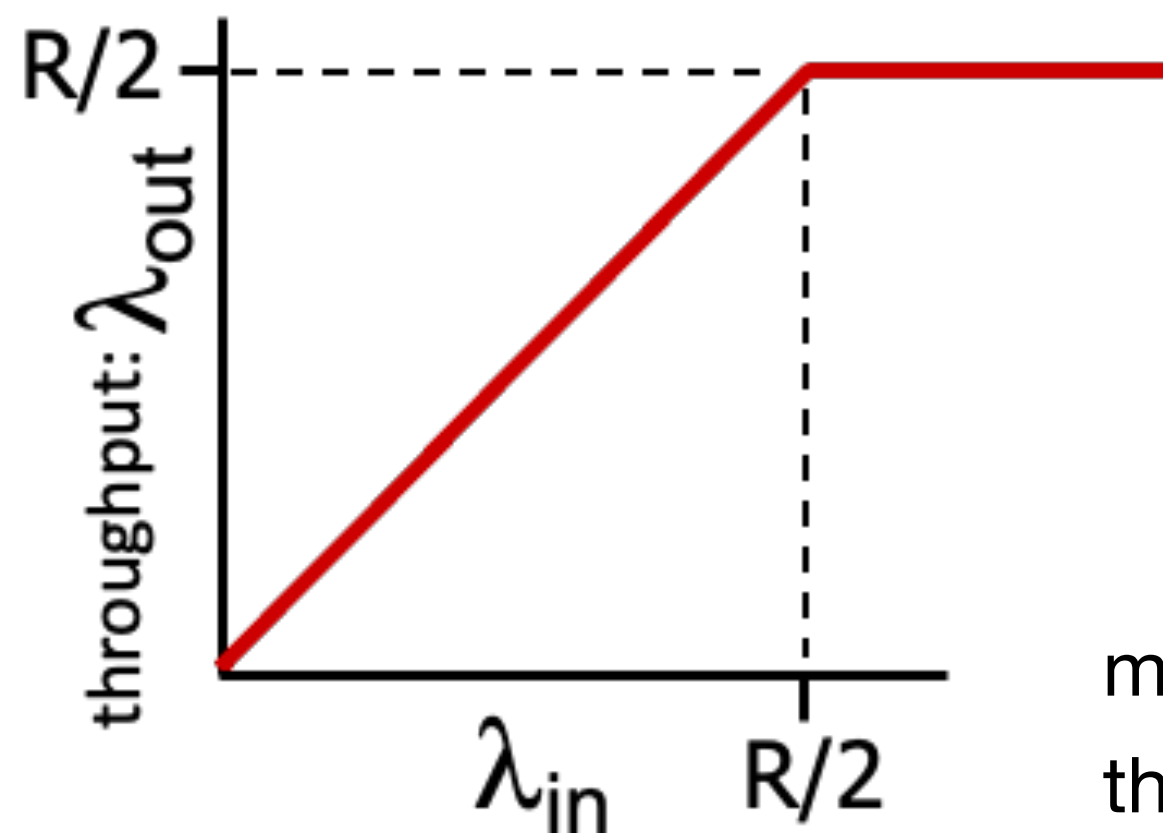
Congestion control:
Too many senders,
sending too fast

Causes/costs of congestion: scenario 1

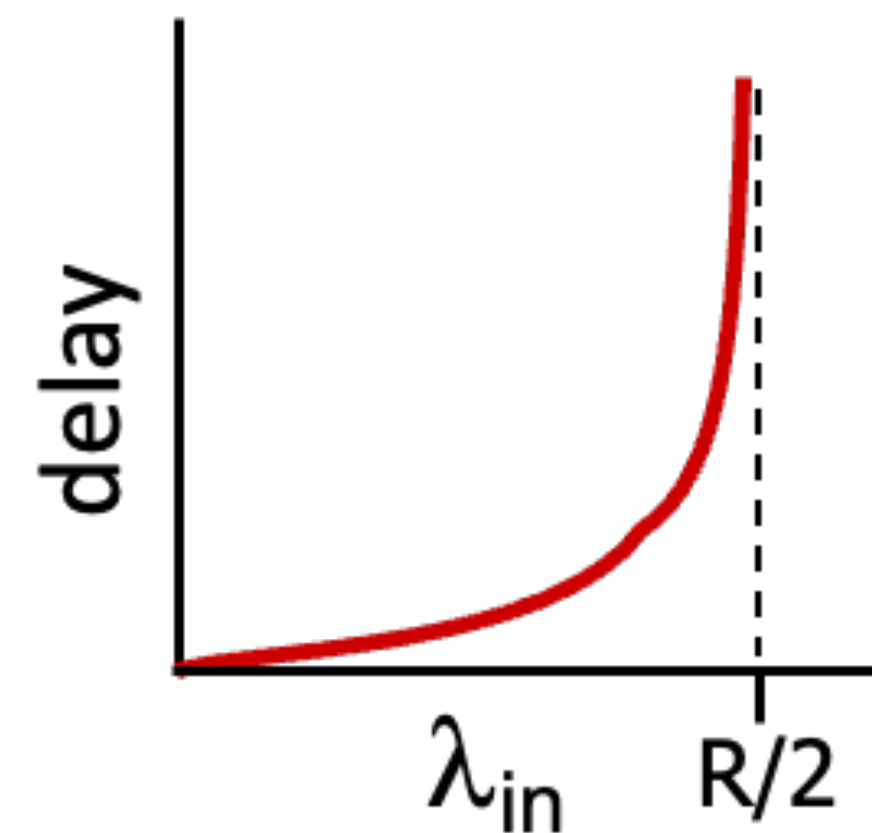
- Simplest scenario
 - One router, infinite buffer
 - Input, output link capacity: R
 - Two flows
 - No retransmission needed



- What happens as arrival rate λ_{in} approaches $R/2$?



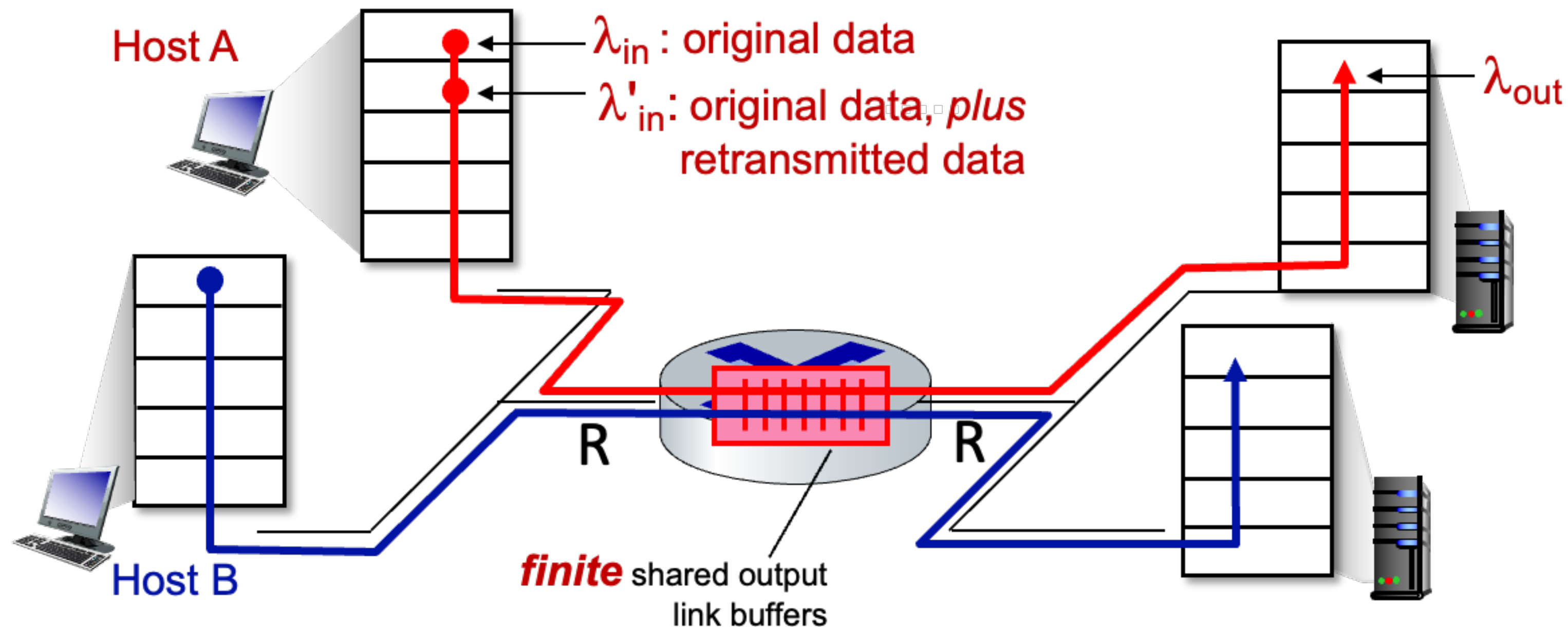
maximum per-connection
throughput: $R/2$



Large delays as arrival rate
 λ_{in} approaches capacity

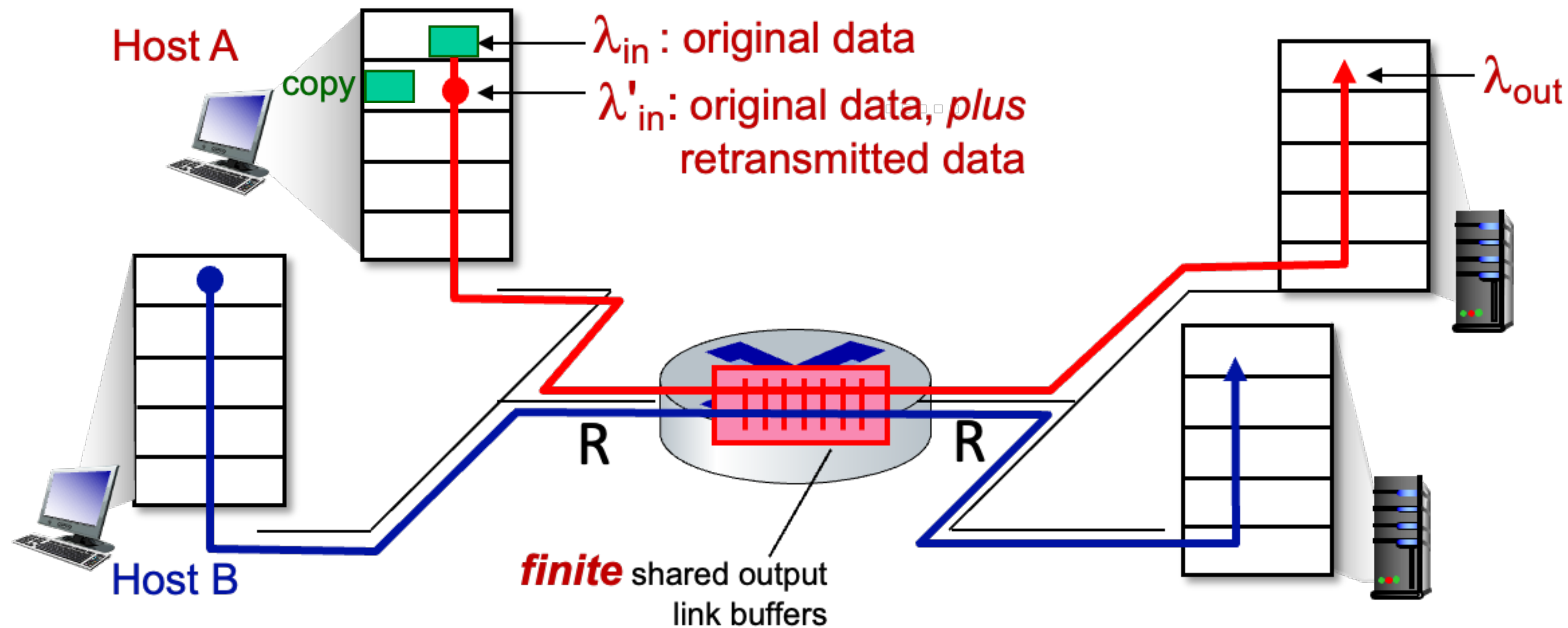
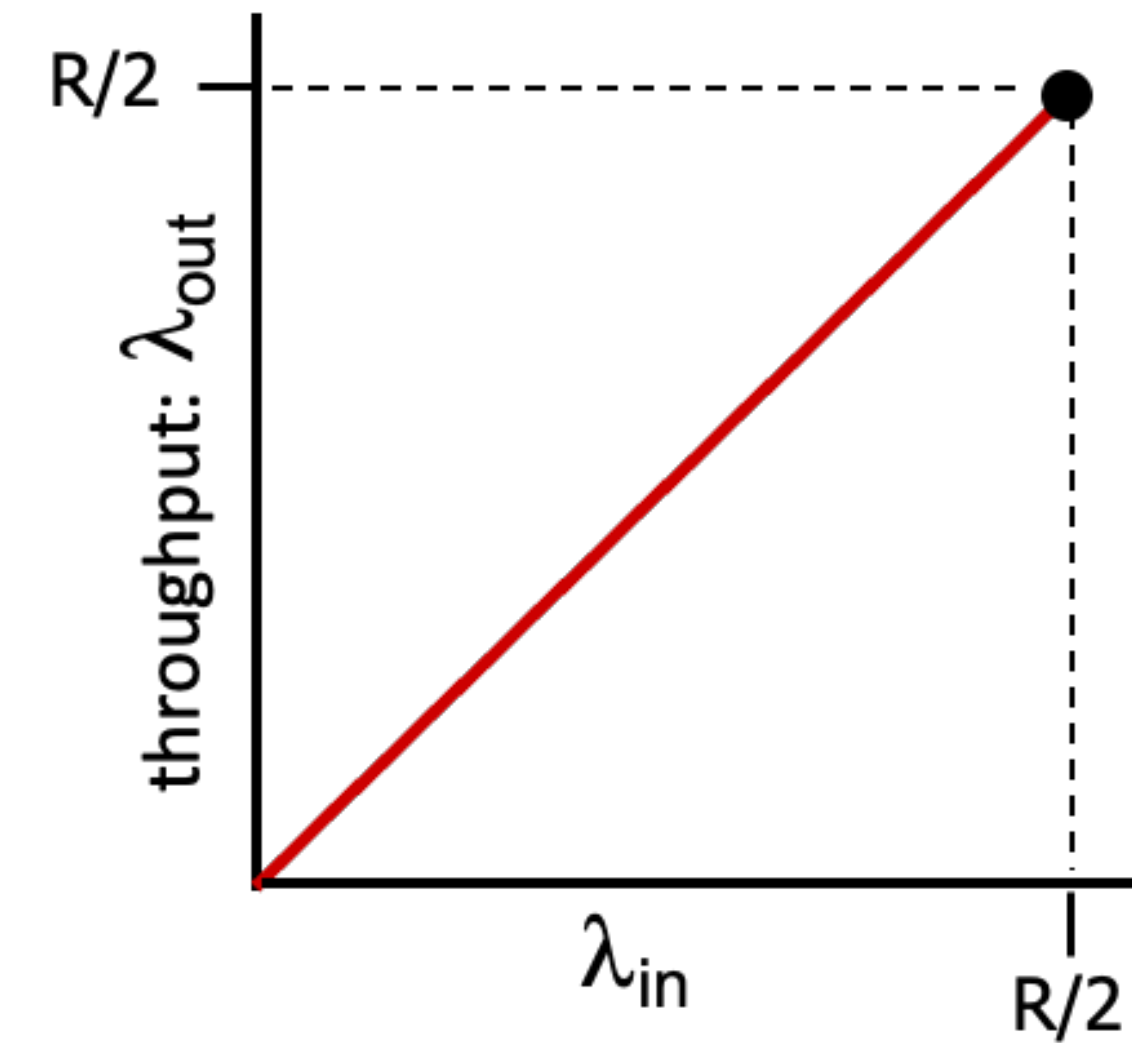
Causes/costs of congestion: scenario 2

- One router, finite buffers
- Sender retransmits lost, timed-out packet
 - application-layer input = application-layer output ($\lambda_{in} = \lambda_{out}$)
 - Transport-layer input includes retransmissions ($\lambda_{in} \geq \lambda_{out}$)



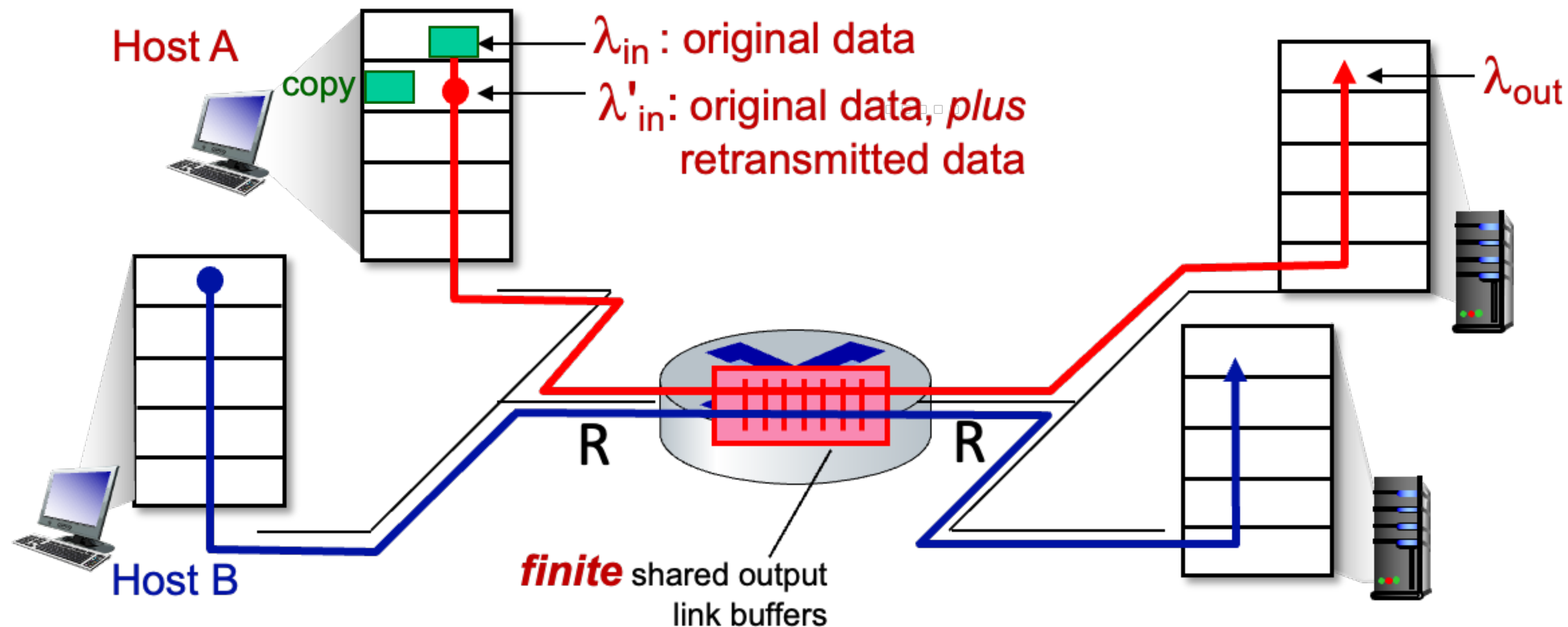
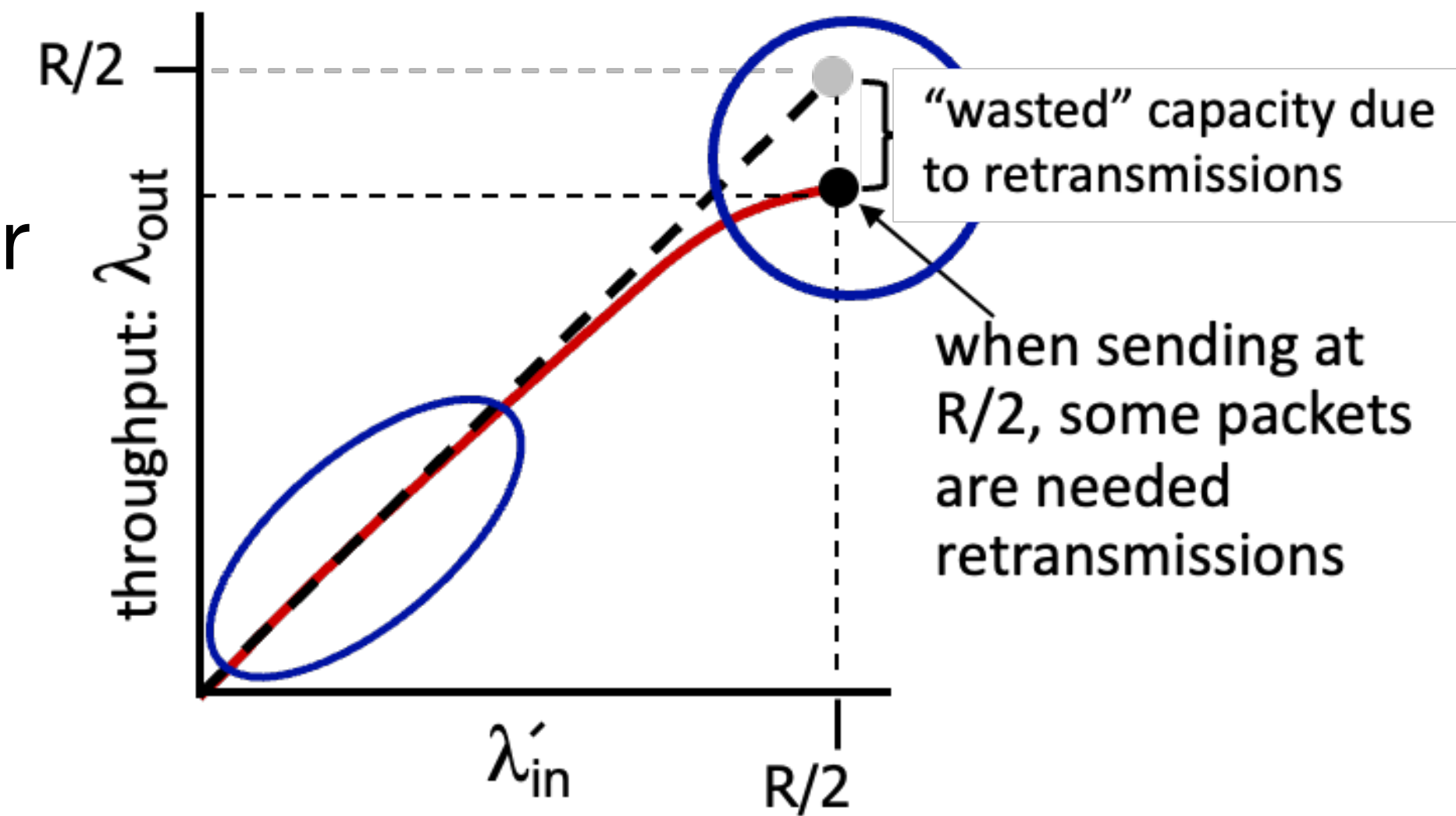
Causes/costs of congestion: scenario 2

- Idealization: perfect knowledge
 - Sender sends only when router buffers become available



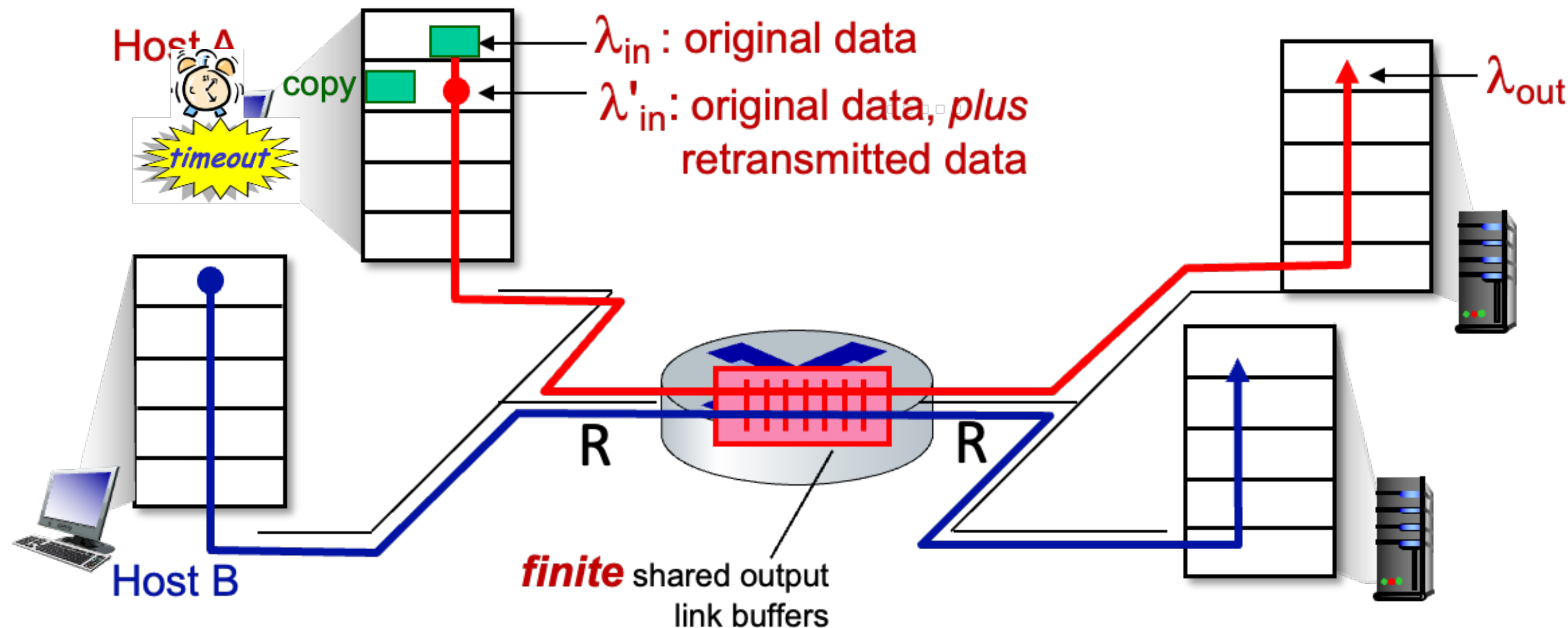
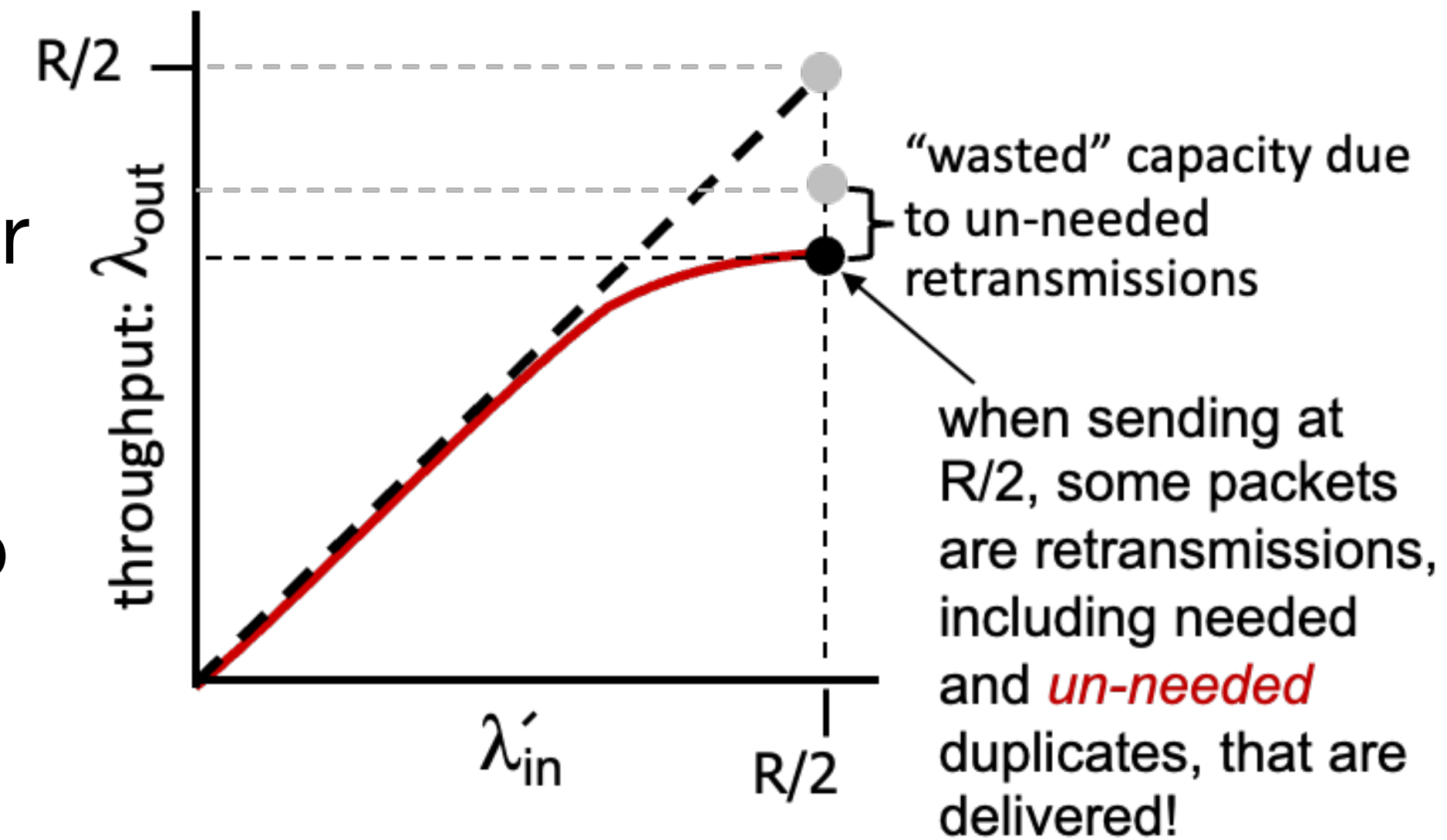
Causes/costs of congestion: scenario 2

- **Idealization:** **some** perfect knowledge
 - Packets can be lost (dropped at router) due to full buffer
 - Sender knows when packet has been dropped: only resends if packet known to be lost



Causes/costs of congestion: scenario 2

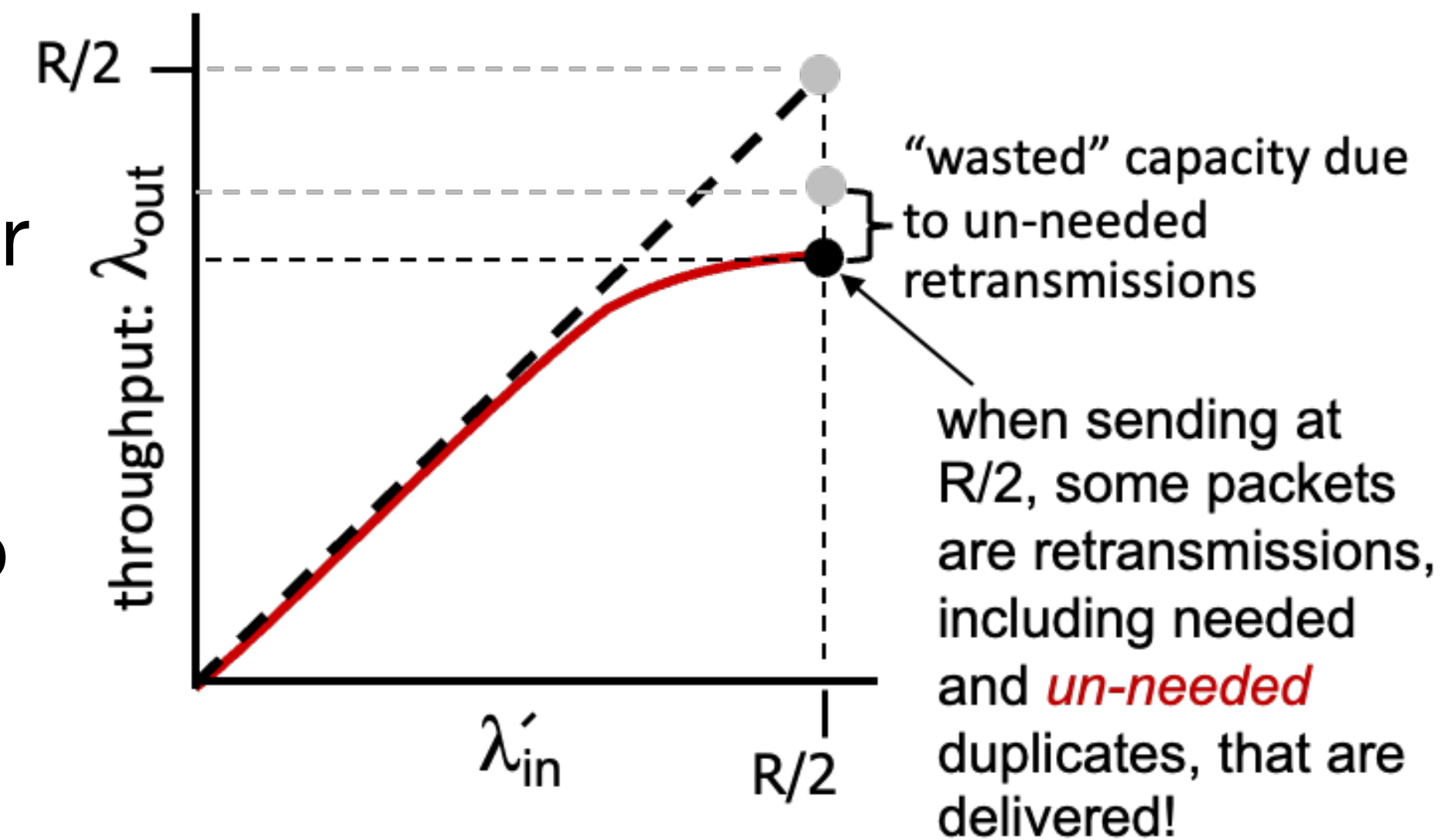
- Realistic scenario: **un-needed duplicates**
 - Packets can be lost (dropped at router) due to full buffer → requiring retransmissions
 - But sender timer can time out prematurely, sending two copies, both of which are delivered



Causes/costs of congestion: scenario 2

- Realistic scenario: **un-needed duplicates**

- Packets can be lost (dropped at router) due to full buffer
→ requiring retransmissions
- But sender timer can time out prematurely, sending two copies, both of which are delivered

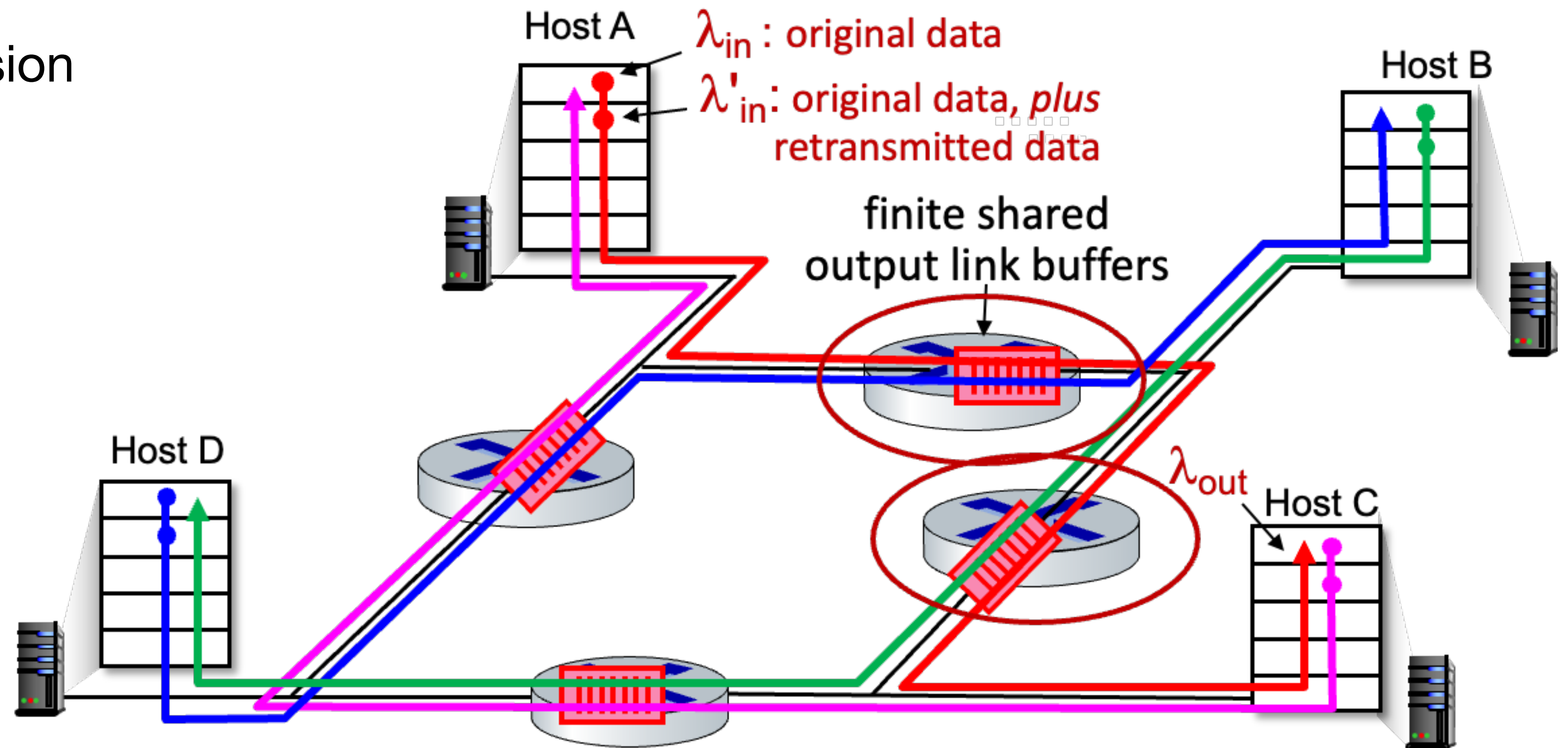


- **Costs of congestion (in summary)**

- More work (retransmission) for given receiver throughput
- Unneeded retransmissions: link carries multiple copies of a packet, decreasing maximum achievable throughput

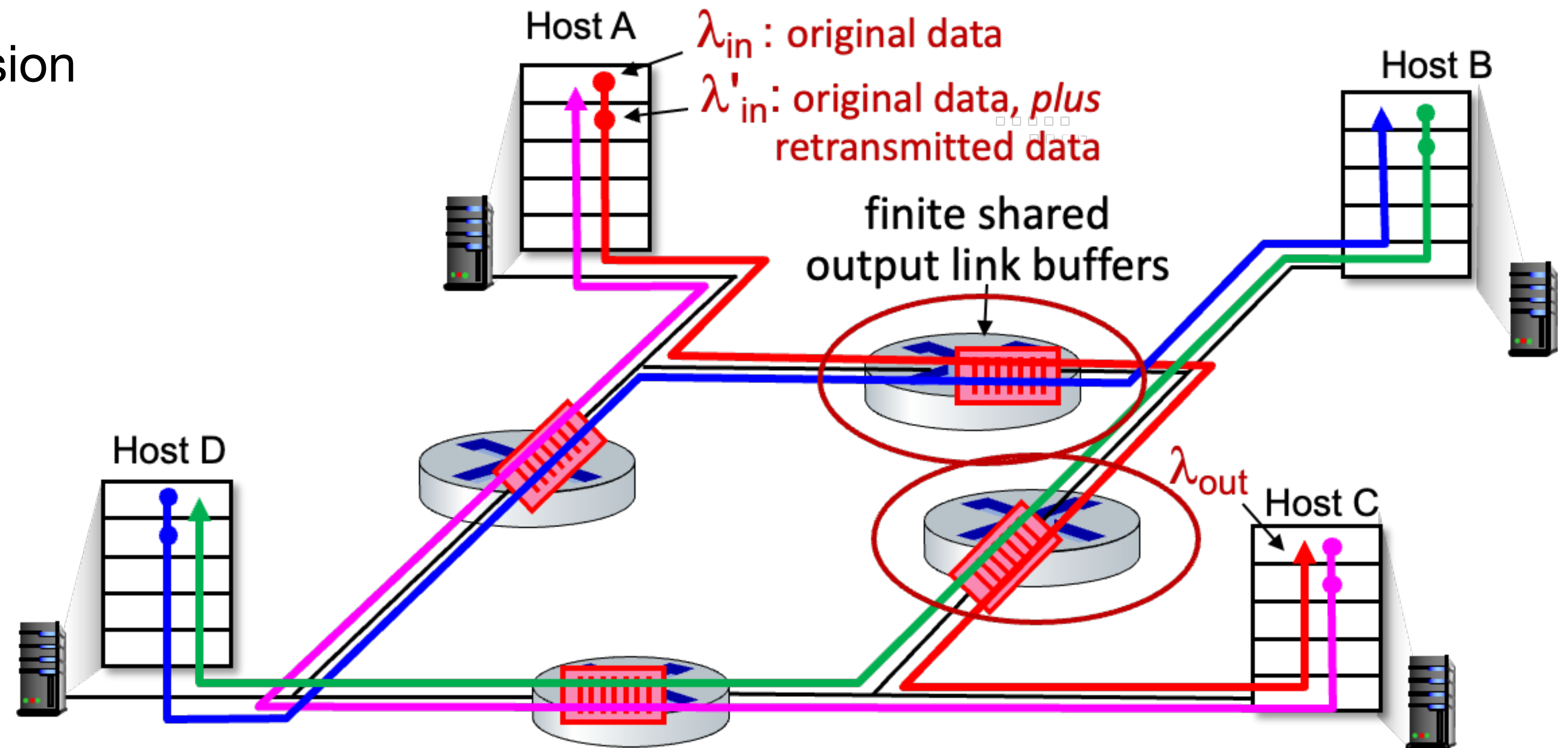
Causes/costs of congestion: scenario 3

- What happens as λ_{in} and λ'_{in} increase?
 - Four senders
 - Multi-hop paths
 - Timeout / retransmission



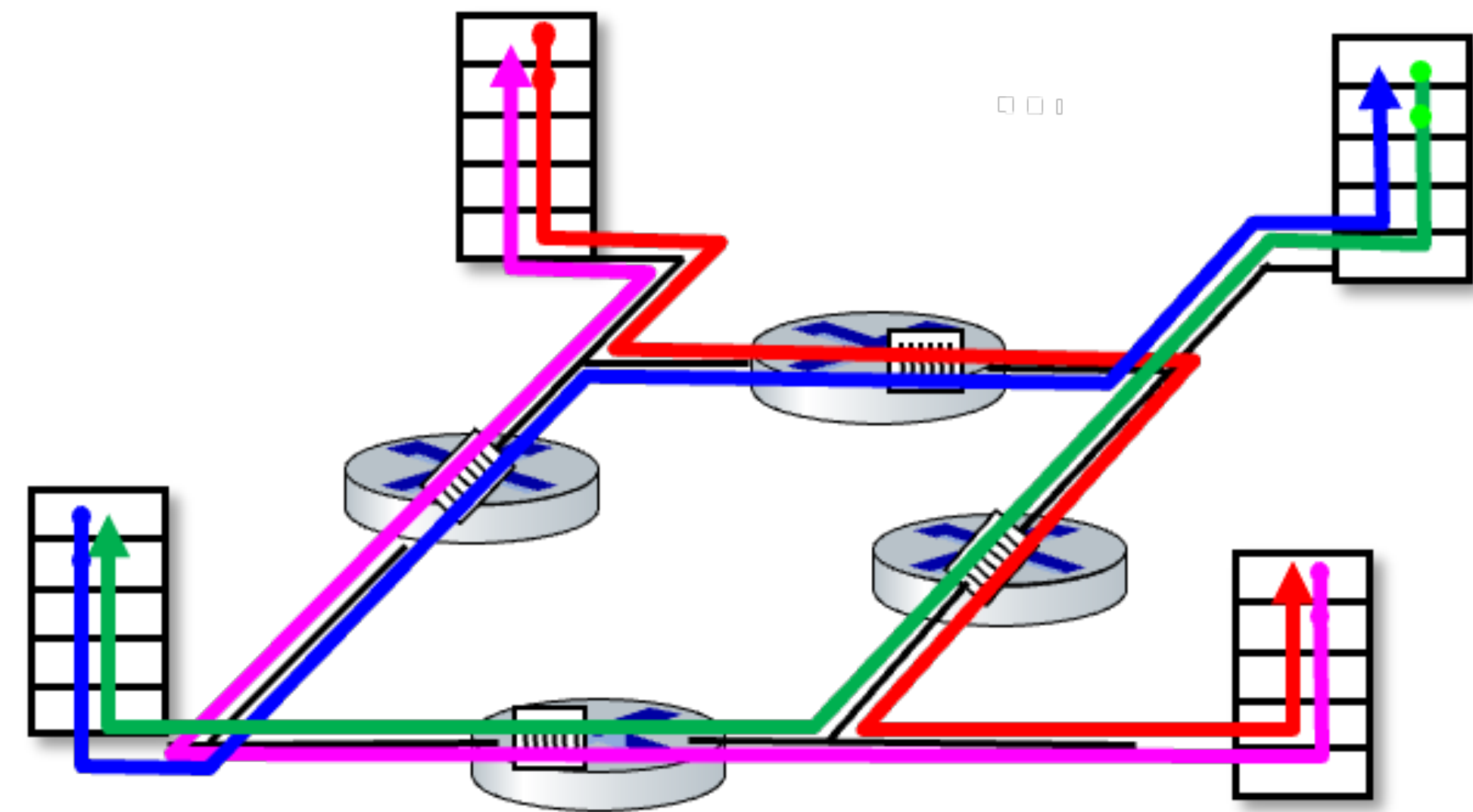
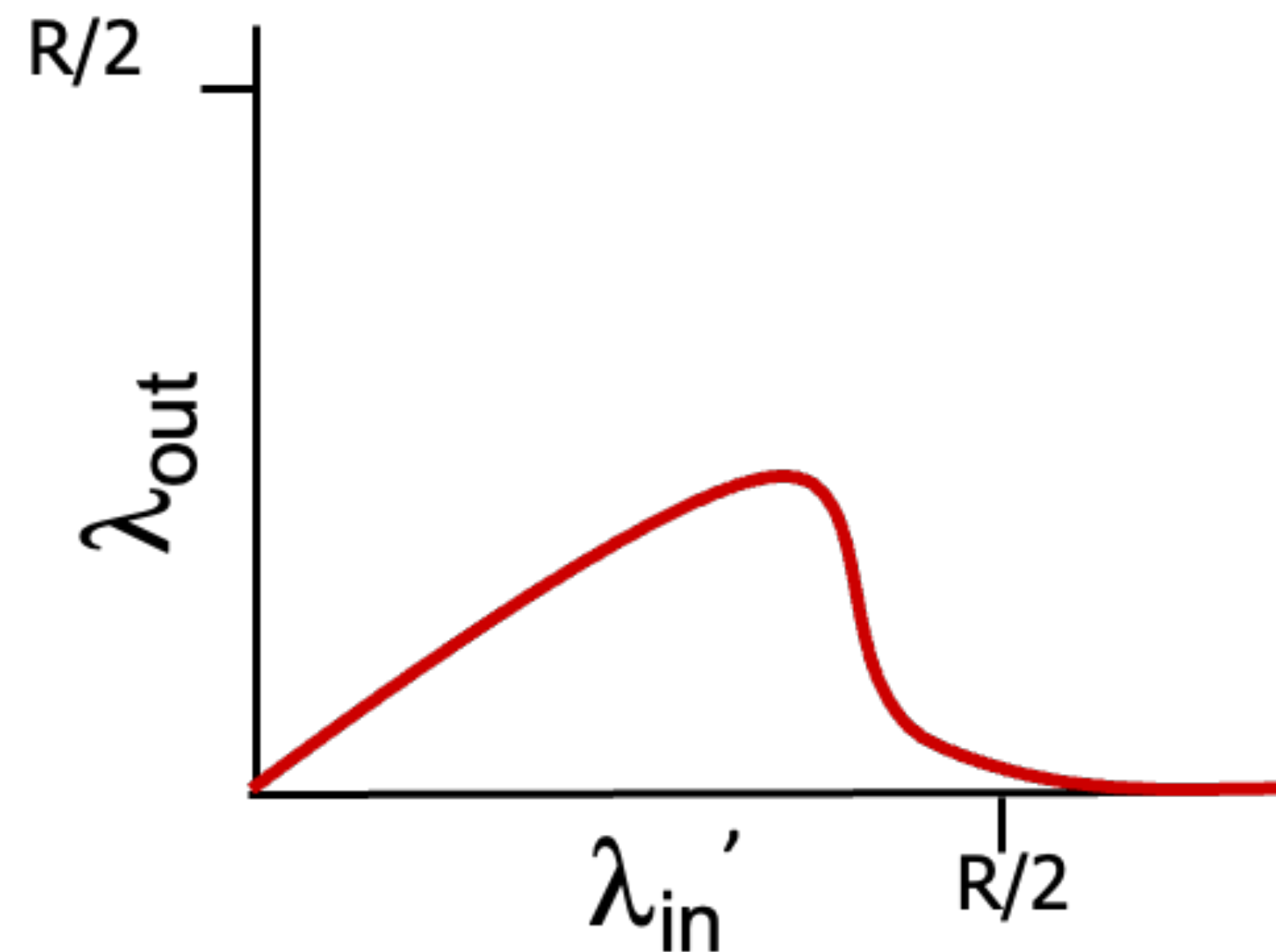
Causes/costs of congestion: scenario 3

- What happens as λ_{in} and λ'_{in} increase?
 - Four senders
 - Multi-hop paths
 - Timeout / retransmission
- ⇒ As red λ'_{in} increases, all arriving blue packets at upper queue are dropped, blue throughput goes to 0



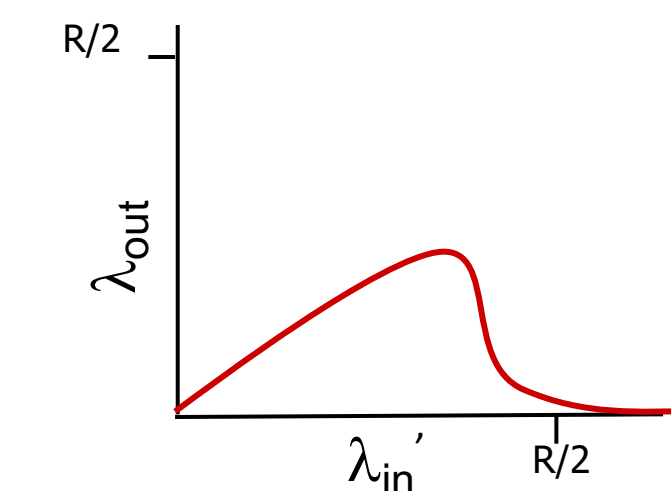
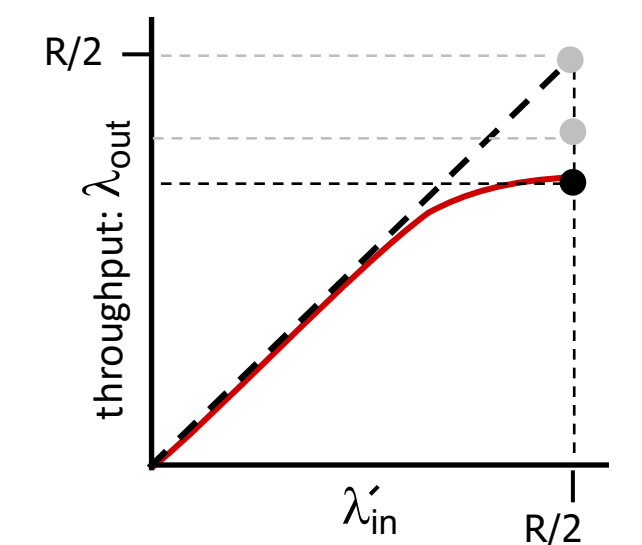
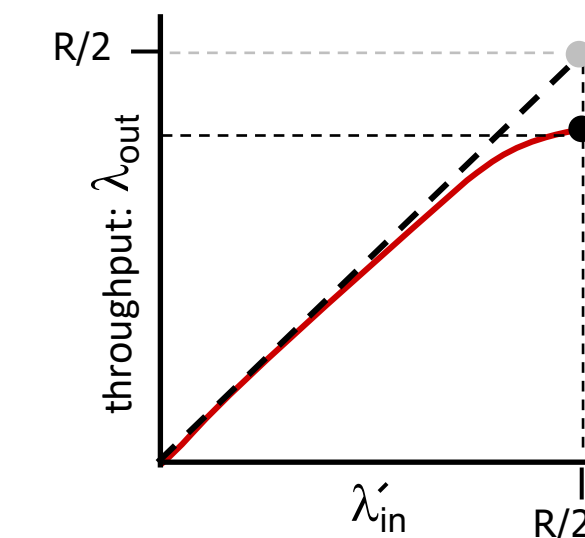
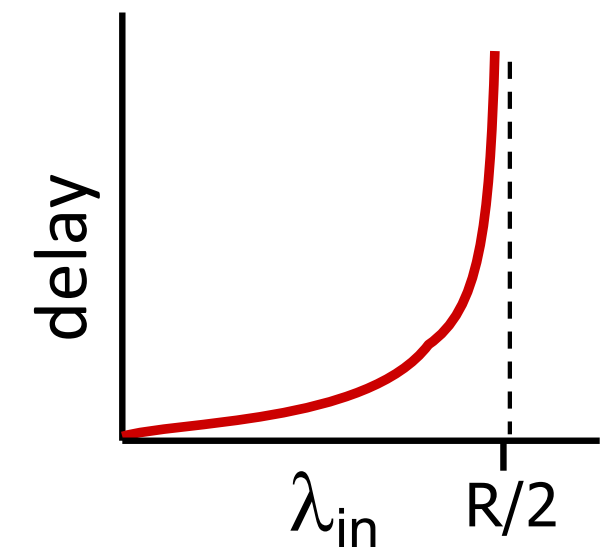
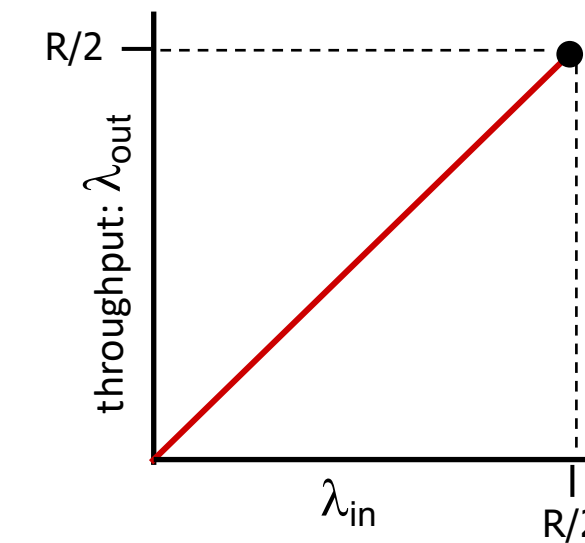
Causes/costs of congestion: scenario 3

- Another cost of congestion
 - When packets are dropped, any upstream transmission capacity and buffering used for that packet was wasted!



Causes/costs of congestion: insights

- Throughput can never exceed capacity
- Delay increases as the traffic amount approaches capacity
- Loss / retransmission decreases effective throughput
- Un-needed duplicates further decreases effective throughput
- Upstream transmission capacity / buffering is wasted for packets which were lost at the downstream

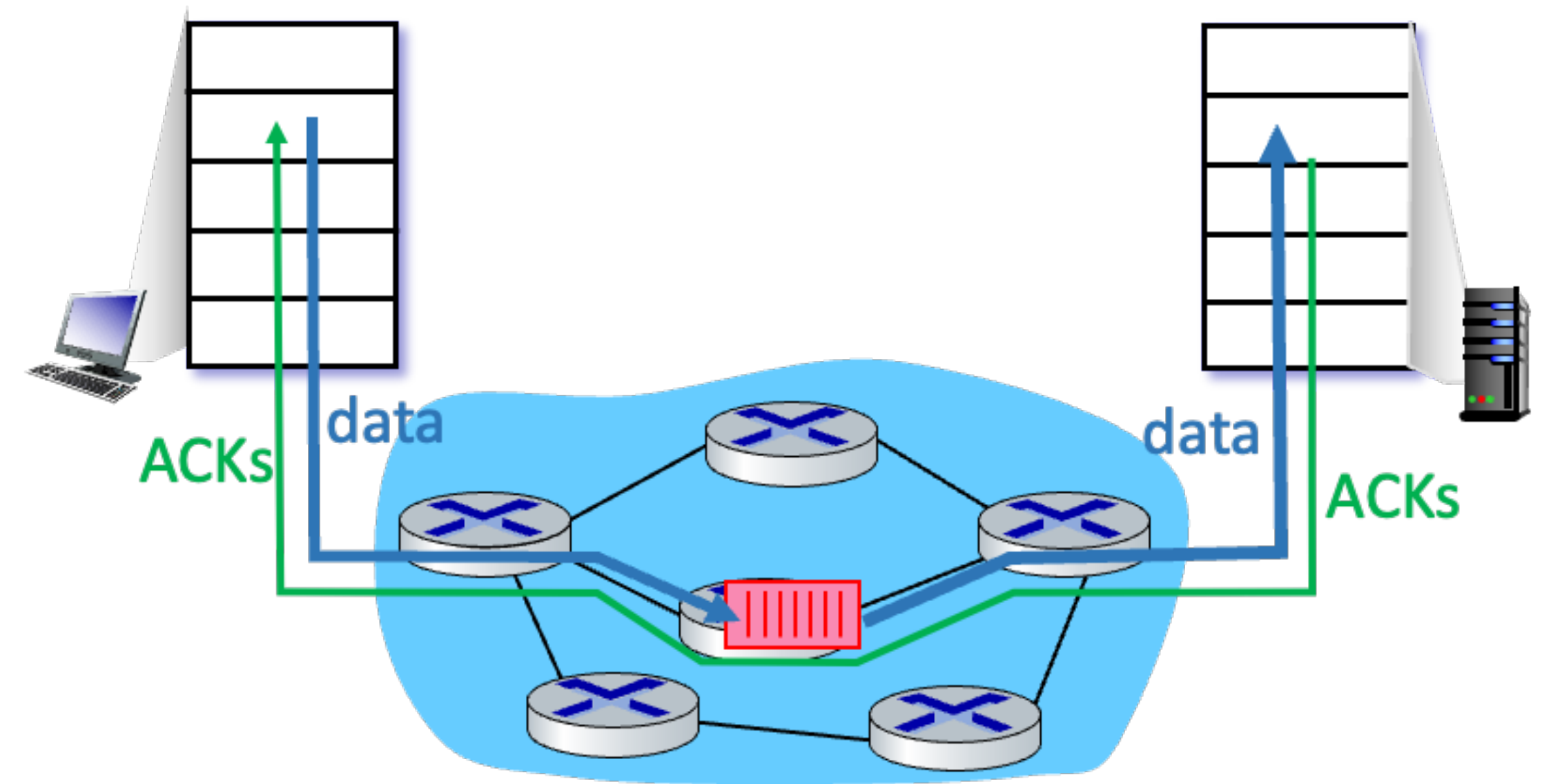


Detailed issues of congestion control

- How to set transmission rate (considering utilization & fairness)
- How to respond to congestion
 - How to detect congestion
 - How to adjust transmission rate

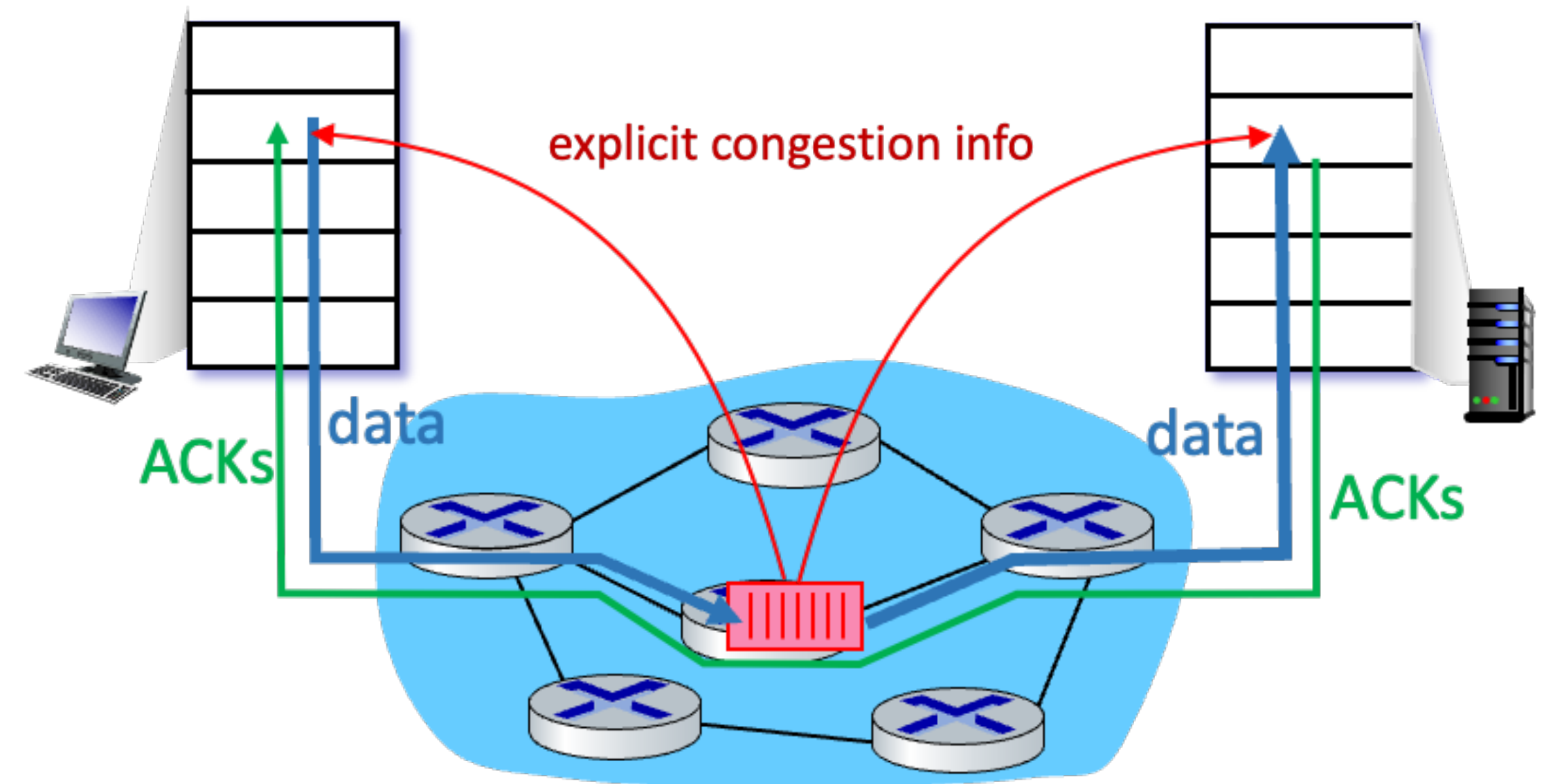
Approaches toward congestion control

- End-to-end congestion control
 - No explicit feedback from network is used
 - Congestion is inferred from observed loss, delay
 - This approach is taken by TCP



Approaches toward congestion control

- Network-assisted congestion control
 - Routers provide direct feedback to sending/receiving hosts of flows passing through congested router
 - May indicate congestion level or explicitly set sending rate
 - TCP ECN, ATM, DECbit protocols



TCP congestion control

TCP congestion control

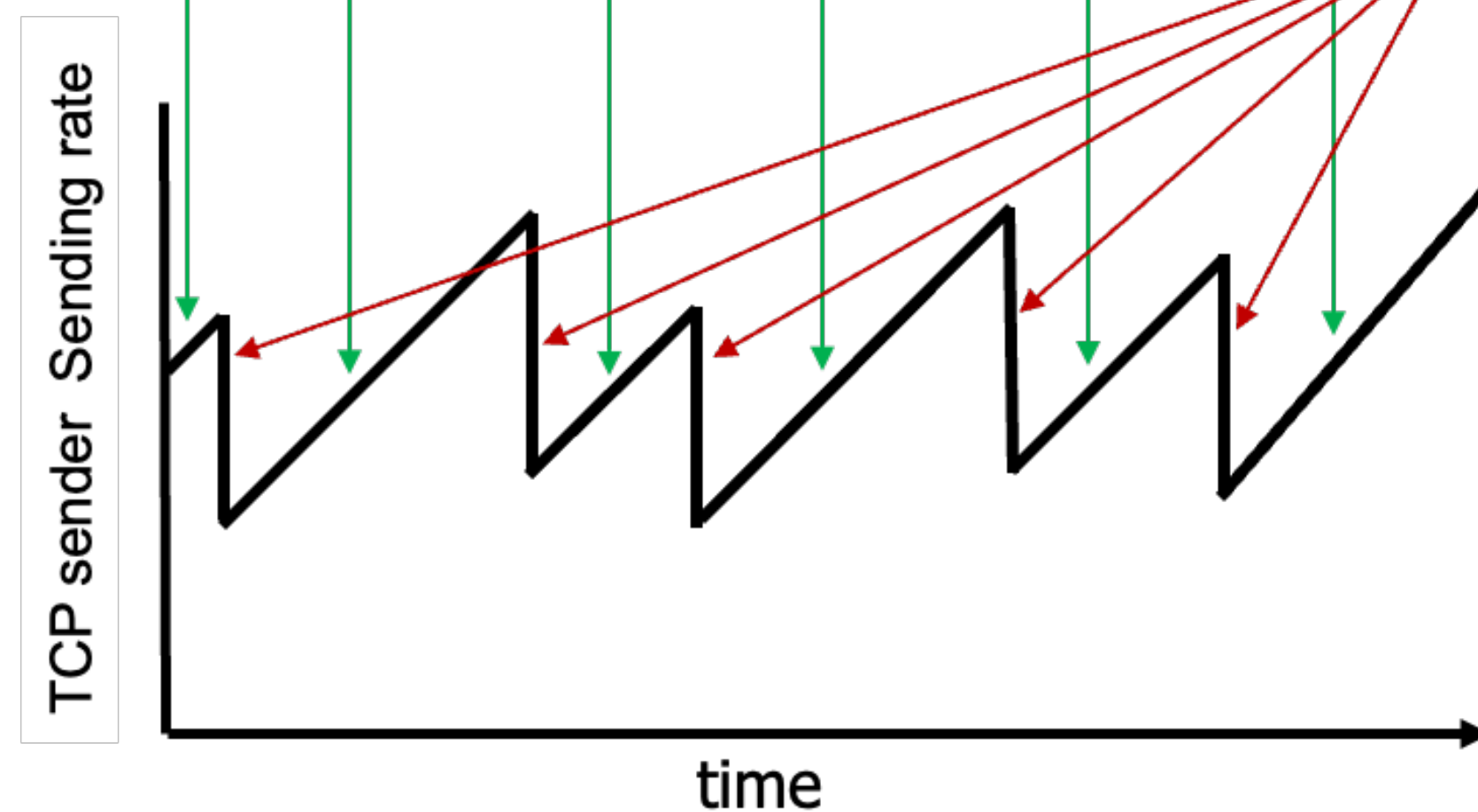
- Approach: senders can increase sending rate until packet loss occurs, then decrease sending rate on loss event

Additive Increase

increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



AIMD sawtooth
behavior: *probing*
for bandwidth

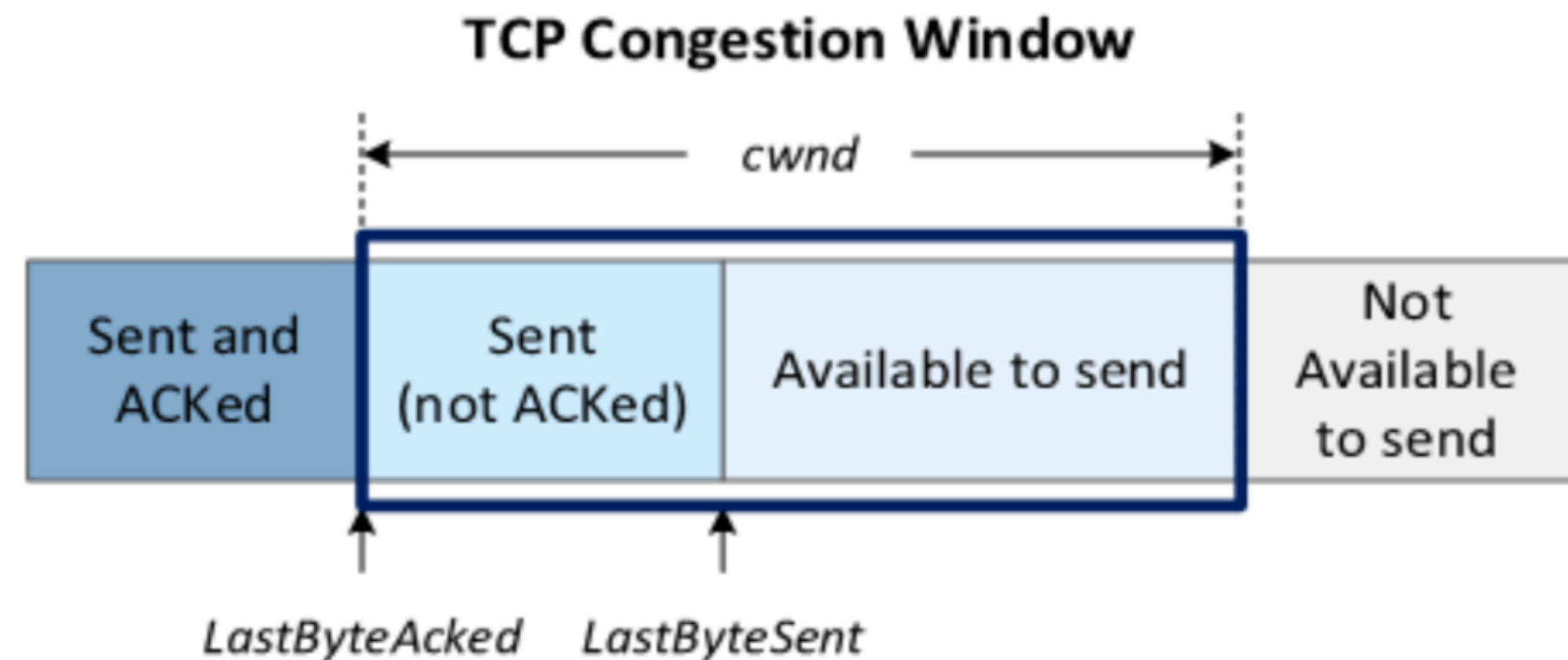
More about TCP AIMD

- Multiplicative decrease detail: sending rate is
 - Cut in half on loss detected by triple duplicate ACKs (TCP Reno)
 - Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)
- Why AIMD?
 - AIMD - a distributed, asynchronous algorithm - has been shown
 - to optimize congested flow rates network wide
 - to has desirable stability properties

TCP congestion control: details

- TCP sending behavior (roughly)
 - Send **CWND** bytes,
 - Wait **RTT** for ACKs
 - Then send more bytes

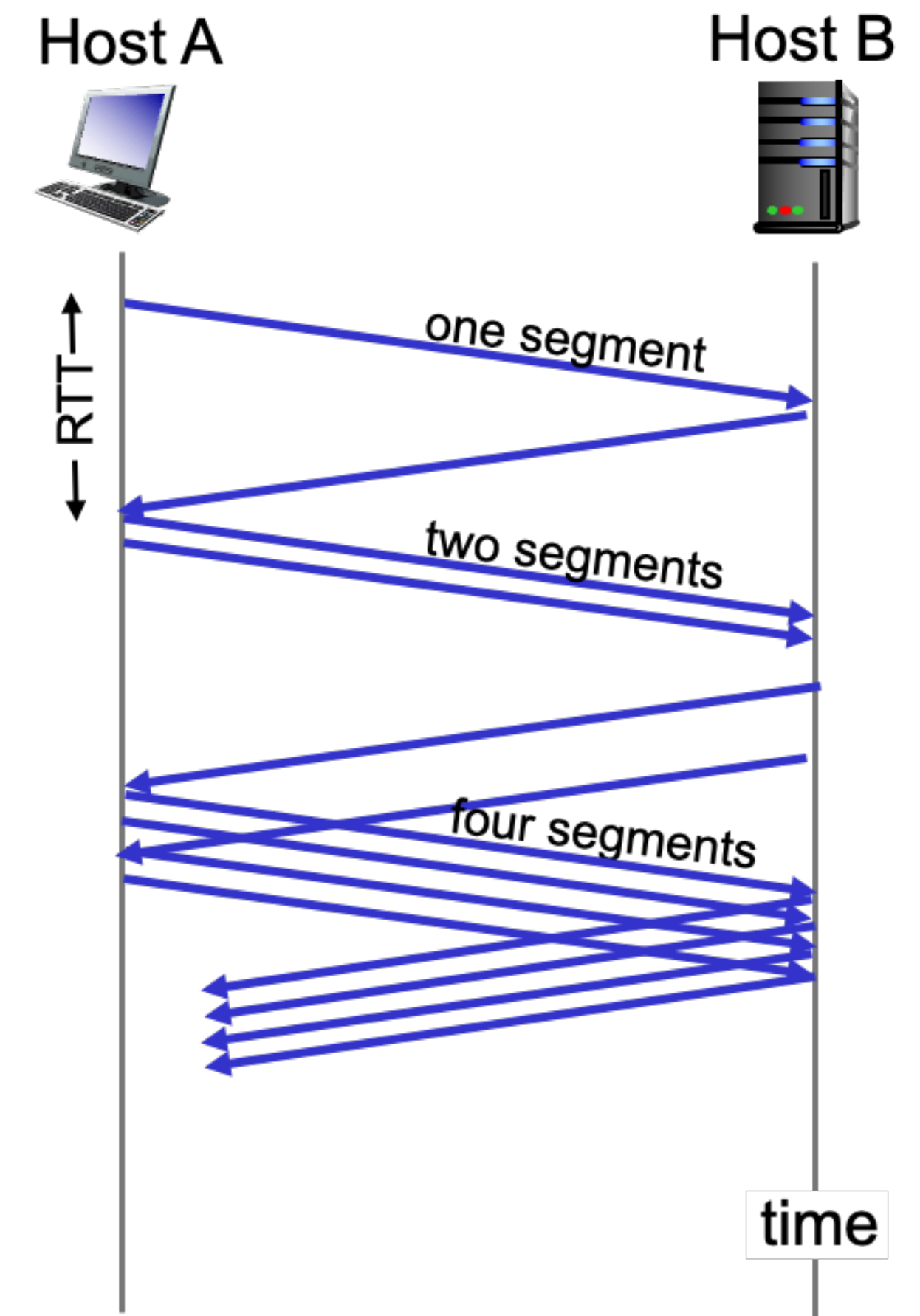
$$\text{TCP rate} \approx \frac{\text{cwnd} \times \text{MSS}}{\text{RTT}} \text{ bytes/sec}$$



- Transmission rate is limited by **CWND**: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{CWND}$
- CWND is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

TCP slow start

- When connection begins, increase rate exponentially until the first loss event occurs
 - Initial **CWND** = 1 MSS
 - For each ACK, $\text{CWND} \leftarrow \text{CWND} + 1$ (packets)
 - Incrementing **CWND** for every ACK received
 - That is, double **CWND** every **RTT**
- Summary
 - Initial rate is slow, but ramps up exponentially fast

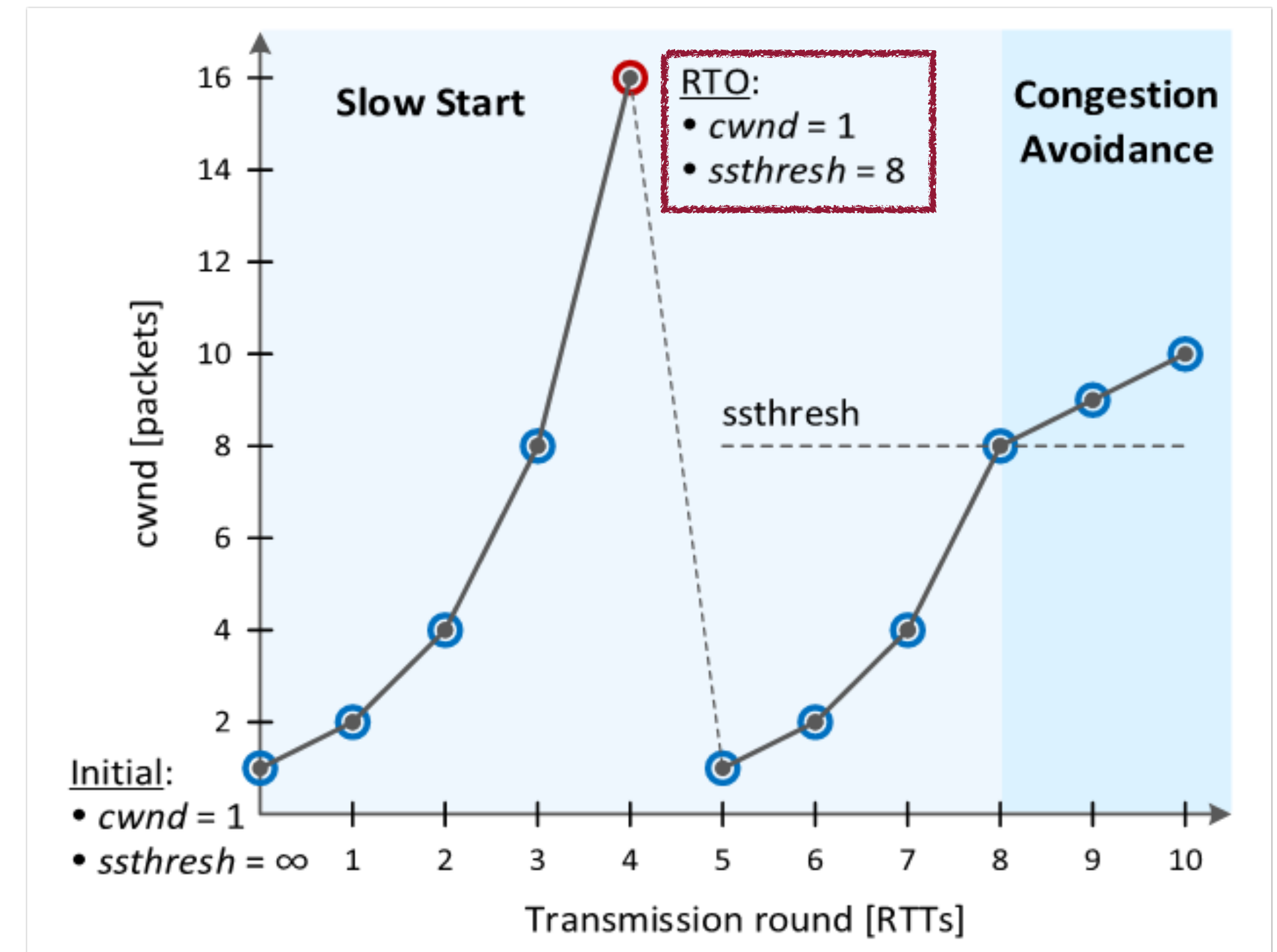


TCP slow start

- When should the exponential increase switch to linear?

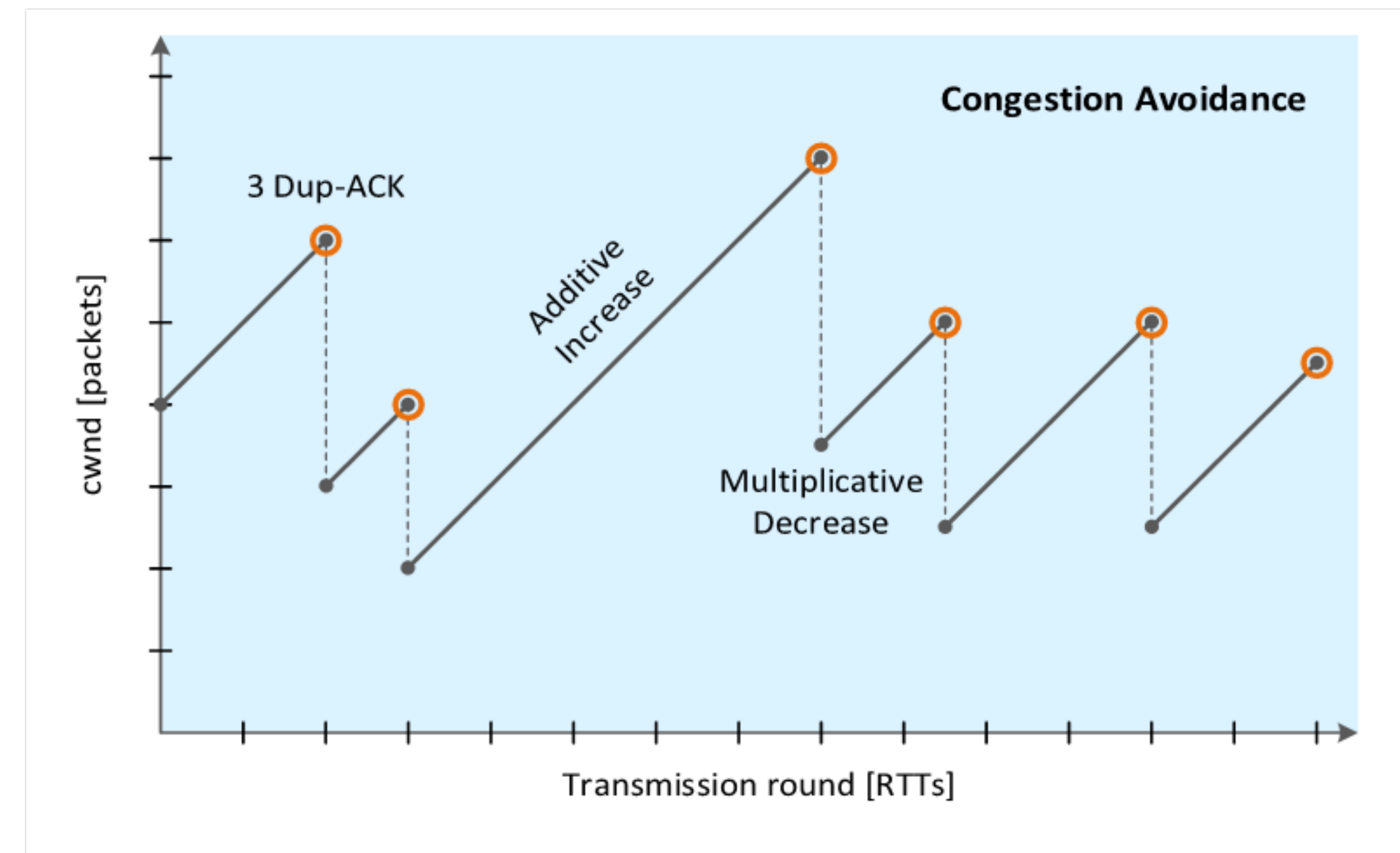
⇒ When CWND gets to 1/2 of its value before timeout

- Implementation
 - Variable **ssthresh**
 - On loss event, **ssthresh** is set to 1/2 of CWND just before loss event



TCP congestion avoidance

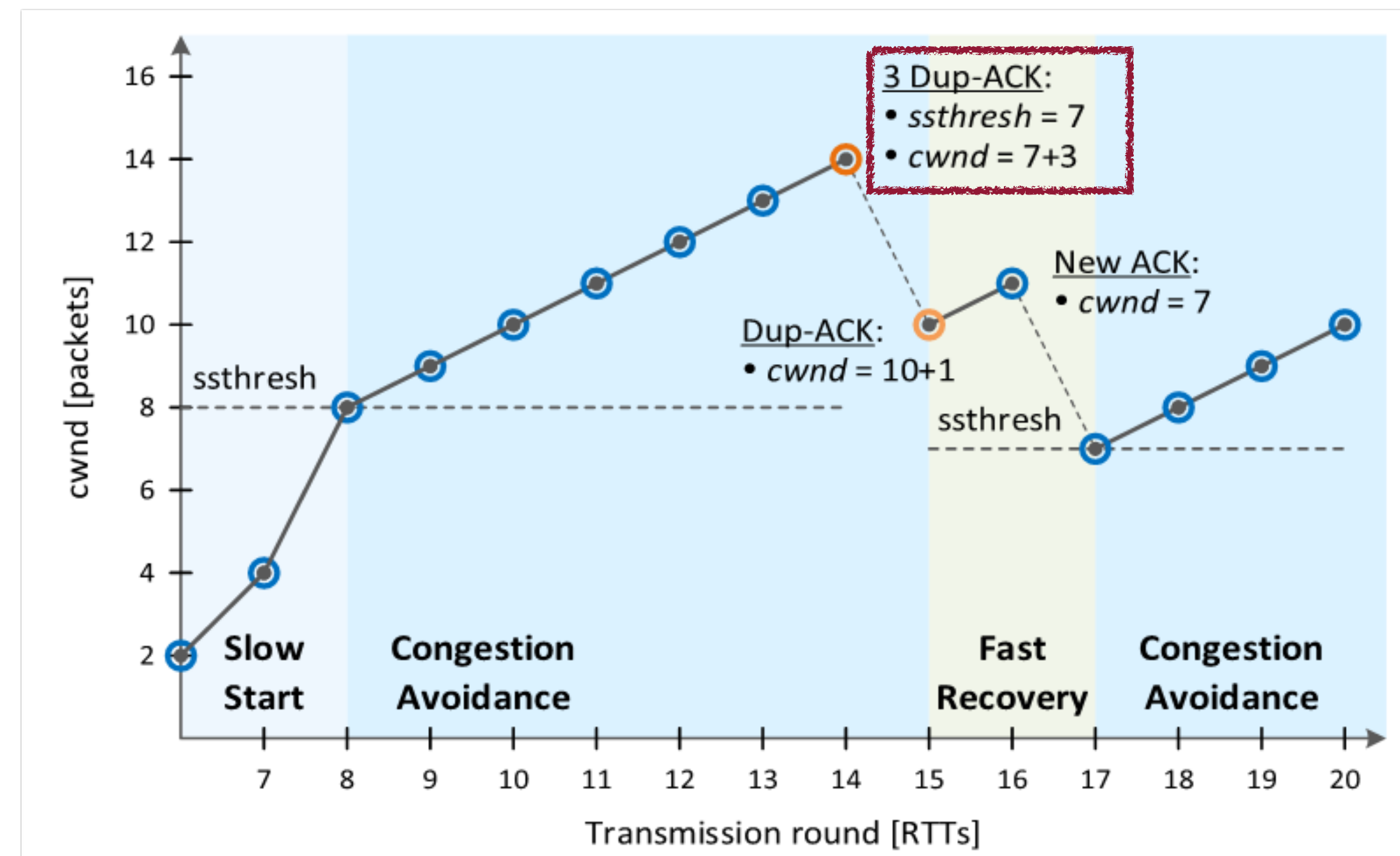
- In congestion avoidance phase (Reno)
 - For each ACK, $\text{CWND} \leftarrow \text{CWND} + 1/\text{CWND}$ (packets)
 - Incrementing **CWND** on every **RTT**
- (Roughly) For each triple duplicate ACKs (loss), $\text{CWND} \leftarrow \text{CWND}/2$
- The detail will be discussed in the next slide



TCP fast recovery

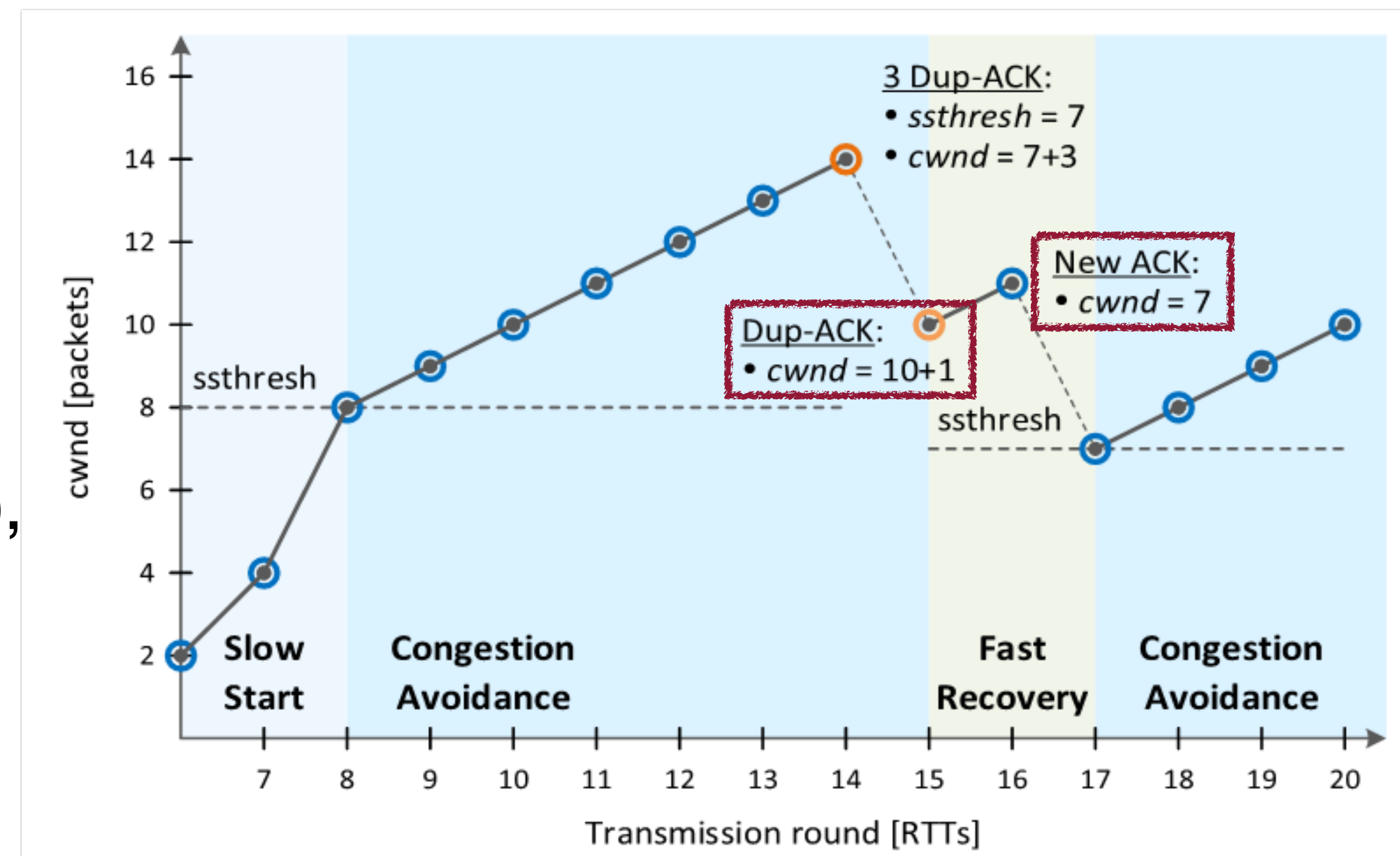
- When receiving triple duplicate ACKs in congestion avoidance phase
 - $ssthresh \leftarrow CWND/2$
 - $CWND \leftarrow ssthresh + 3$

- ▶ Reception of triple duplicate ACKs implies that a packet is lost but subsequent packets are well received
- ▶ But, CWND cannot move forward due to the lost packet
- ▶ 3 is added to secure space in CWND for sending more packets

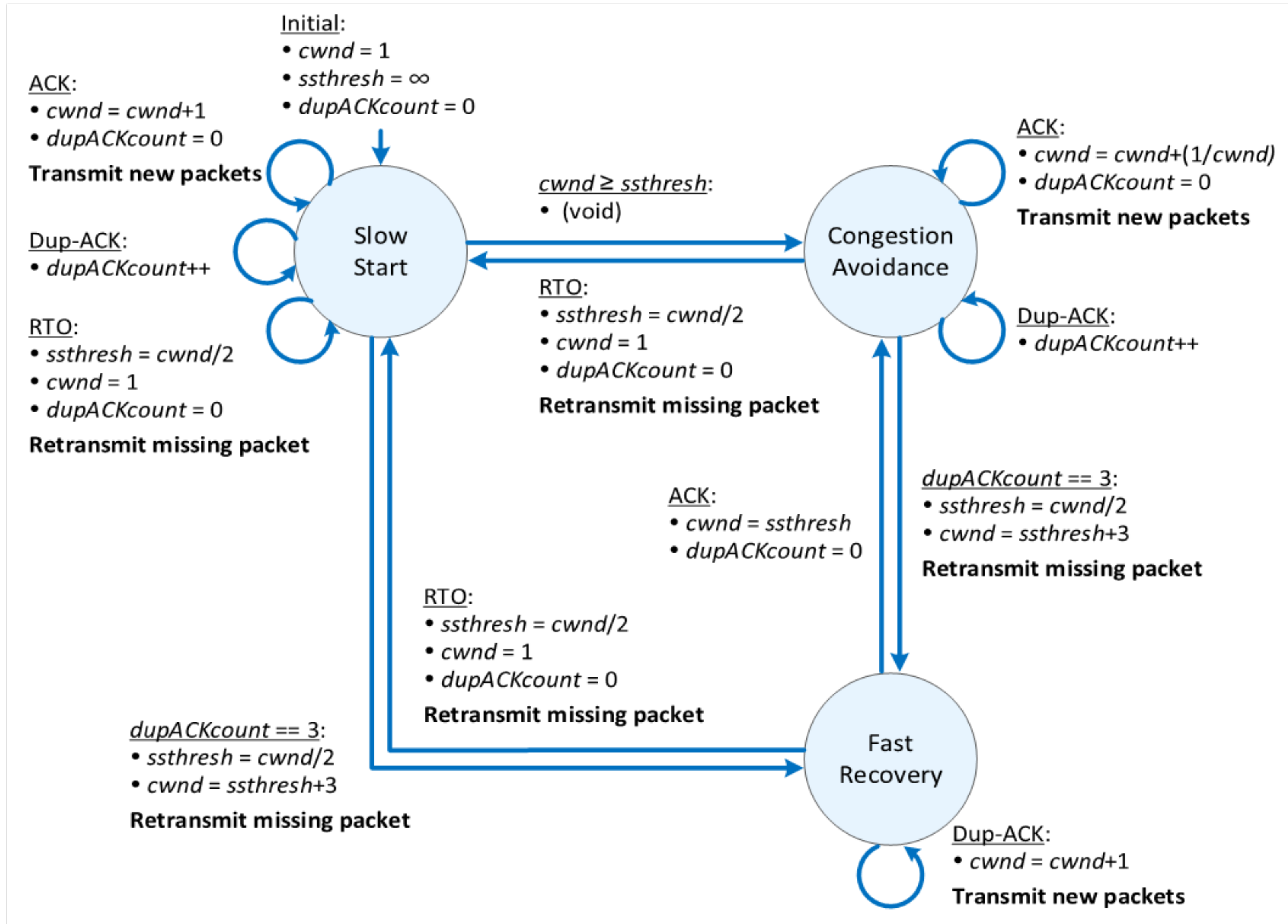


TCP fast recovery

- When receiving triple duplicate ACKs in congestion avoidance phase
 - $ssthresh \leftarrow CWND/2$
 - $CWND \leftarrow ssthresh + 3$
- When receiving more duplicate ACKs
 - For each duplicate ACK,
 - $CWND \leftarrow ssthresh + 1$
 - For each ACK (for retransmission of lost packet),
 - $CWND \leftarrow ssthresh$

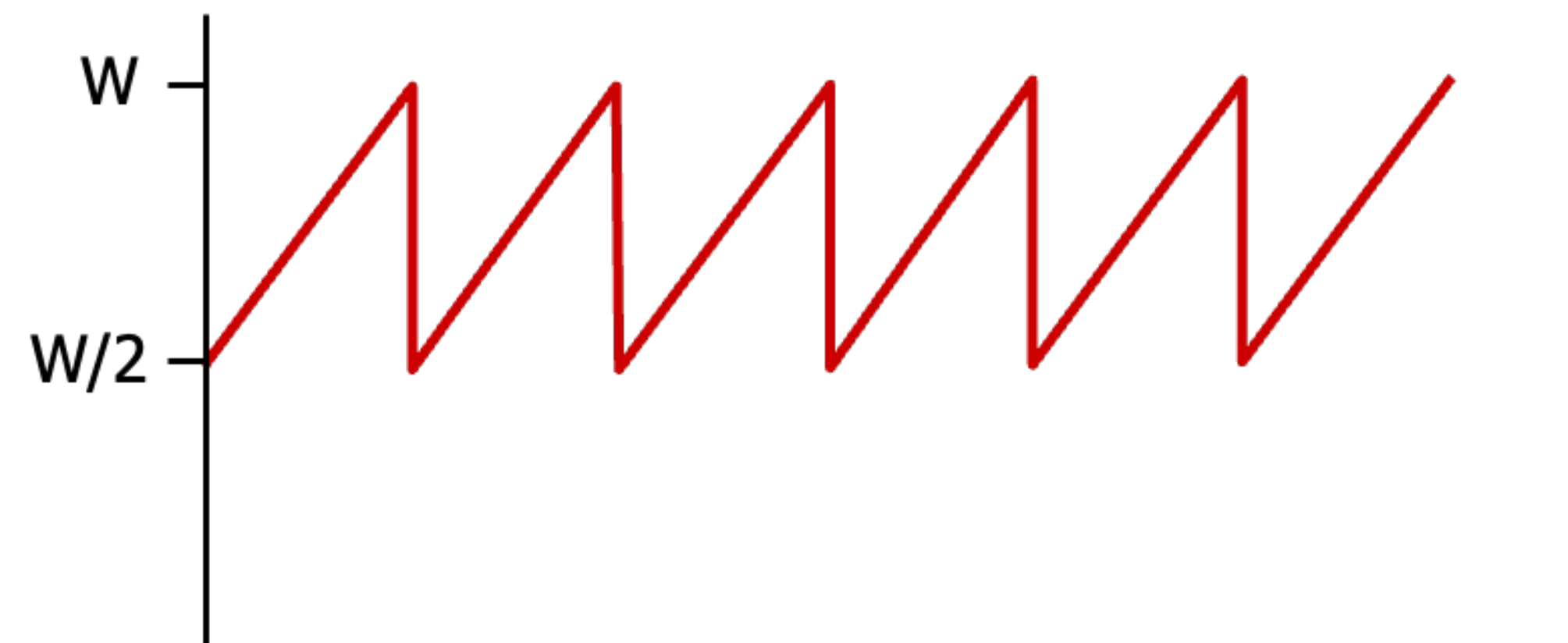


TCP congestion control



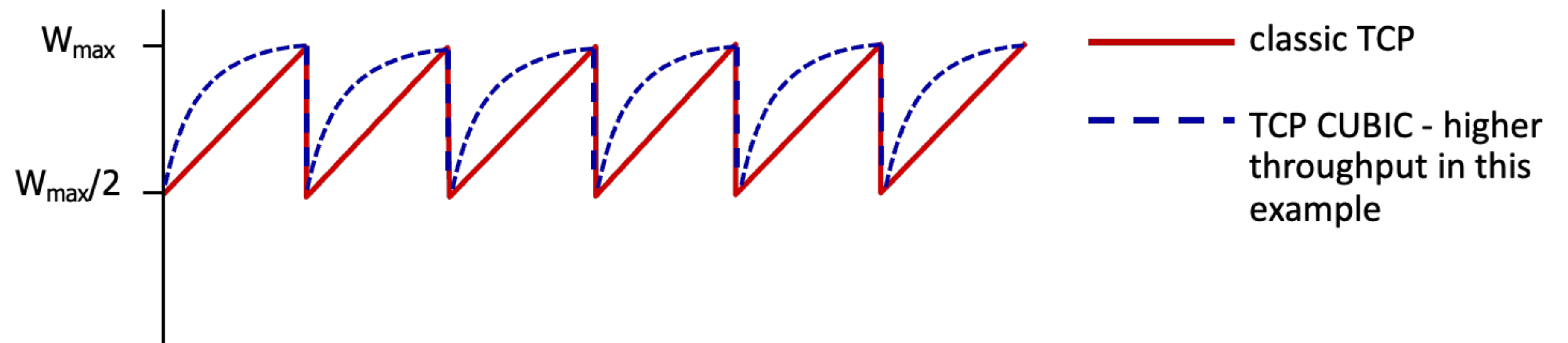
TCP throughput

- Average TCP throughput as a function of window size and RTT
 - Ignore slow start
 - Assume that there is always data to send (saturated condition)
- W (window size measured in byte) where a loss occurs
 - Average window size (# of in-flight bytes) is $\frac{3}{4}W$
 - Average throughput is $\frac{3}{4}W$ per RTT



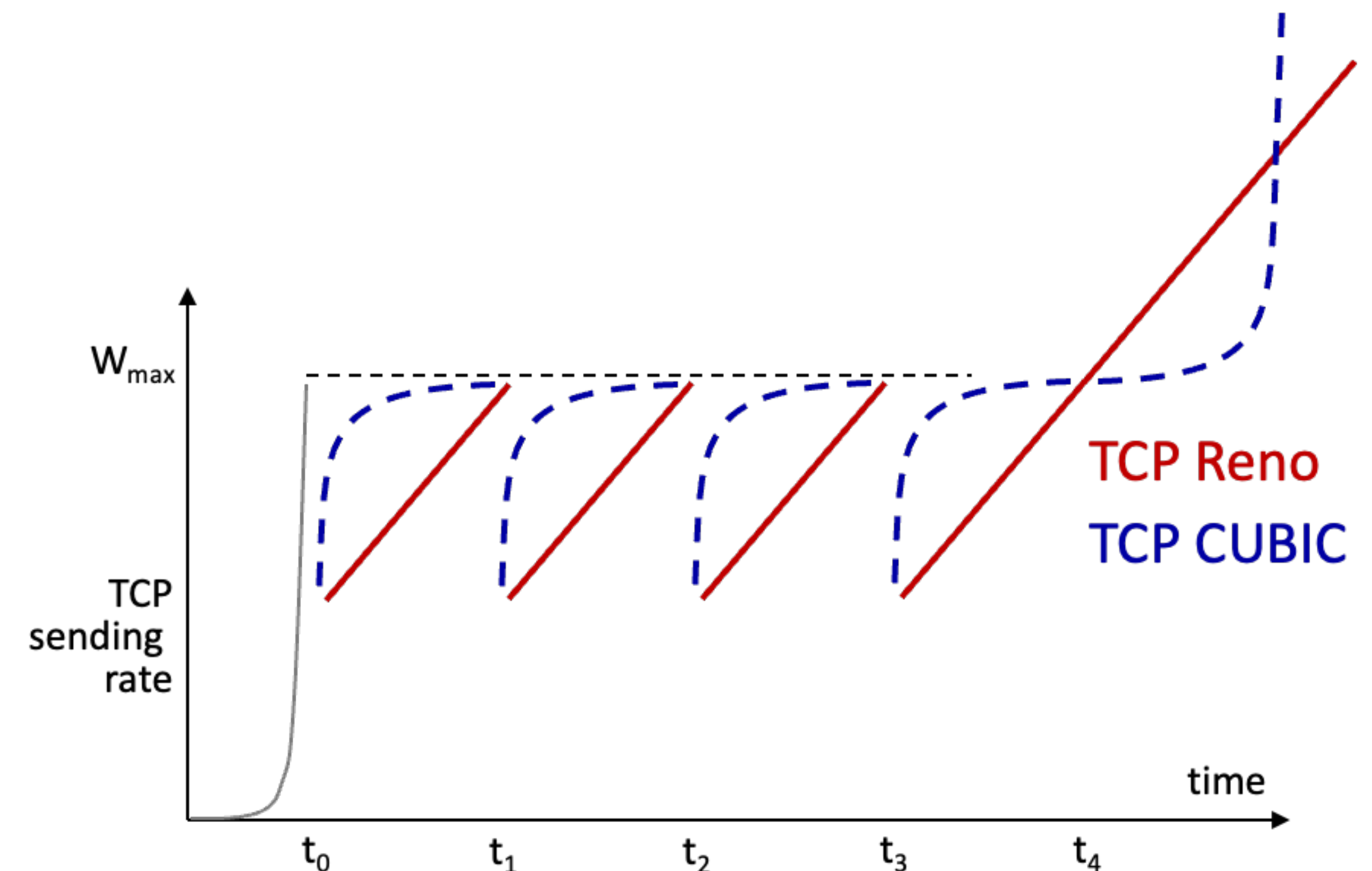
TCP CUBIC

- Is there a better way than AIMD to “probe” for available bandwidth?
- Insight / intuition
 - W_{max} : sending rate at which congestion loss was detected
 - Congestion state of bottleneck link probably(?) has not changed much
 - After cutting rate/window in half on loss, initially ramp W (CWND size) up toward W_{max} faster, but then approach W_{max} more slowly



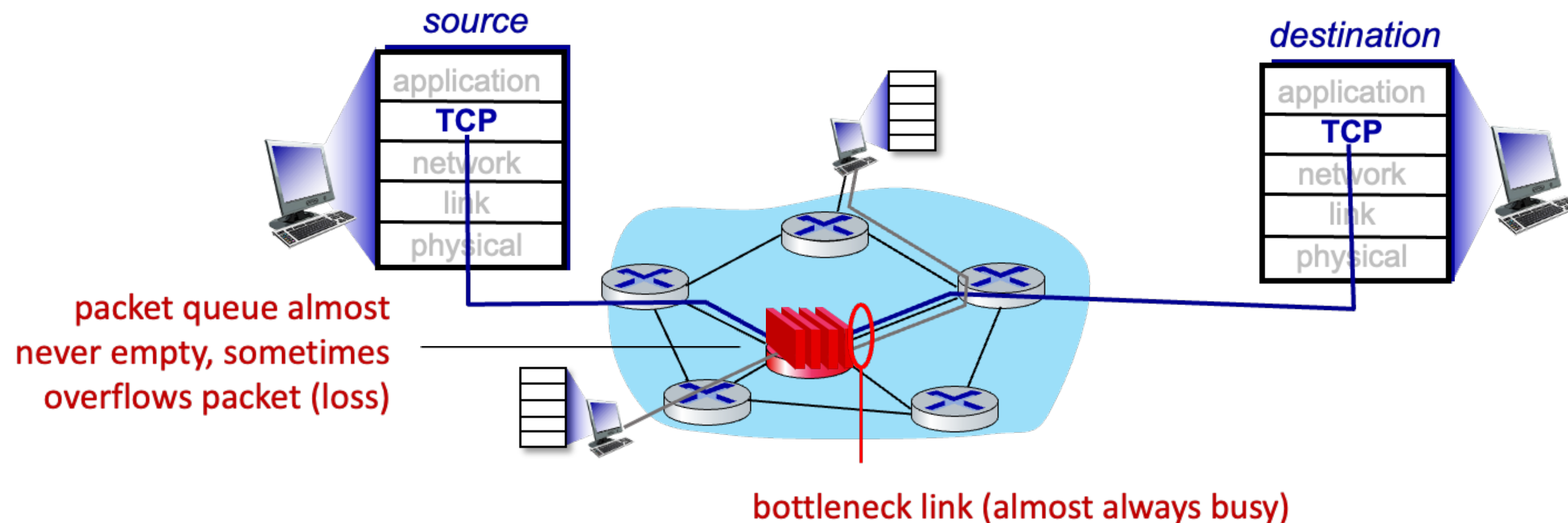
TCP CUBIC

- K : point in time when W will reach W_{max}
 - K itself is tunable
- Increase W as a function of the cube of the distance between current time and K
 - Larger increase when further away from K
 - Smaller (cautious) increase when nearer K
- TCP CUBIC is the default congestion control scheme in LINUX
 - Most popular TCP for popular web servers



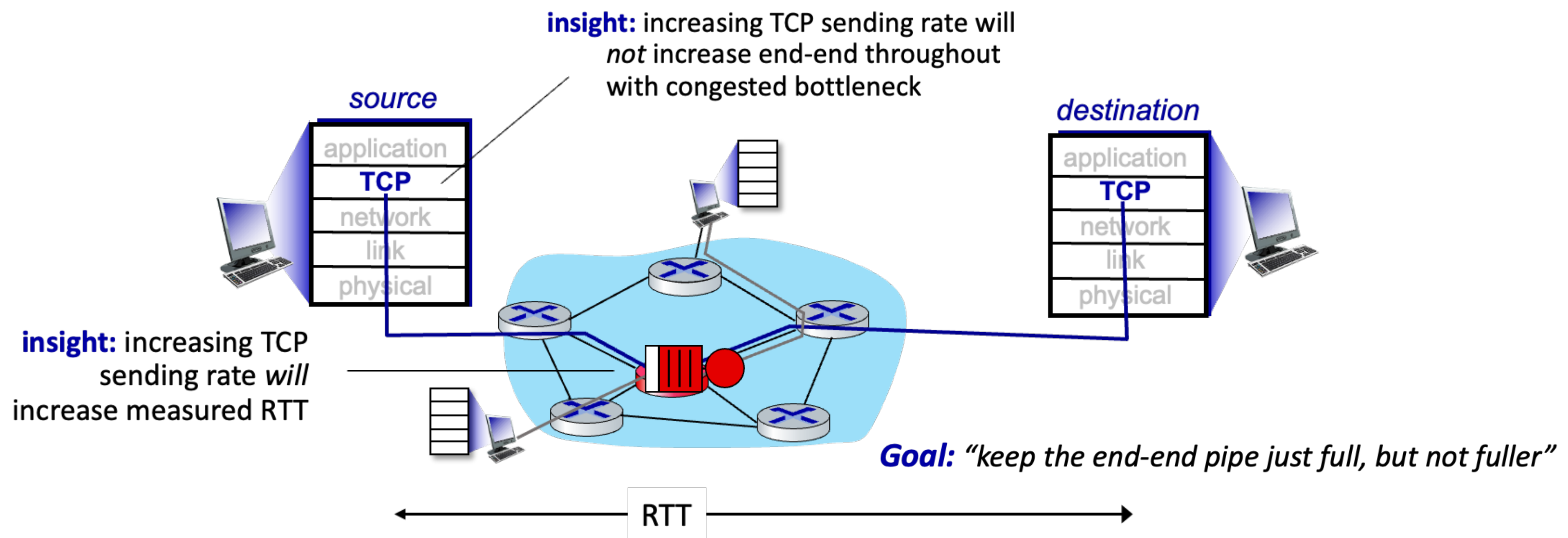
TCP & Congested bottleneck link

- TCP (classic, CUBIC) increases TCP's sending rate until packet loss occurs at some router's output → the bottleneck link



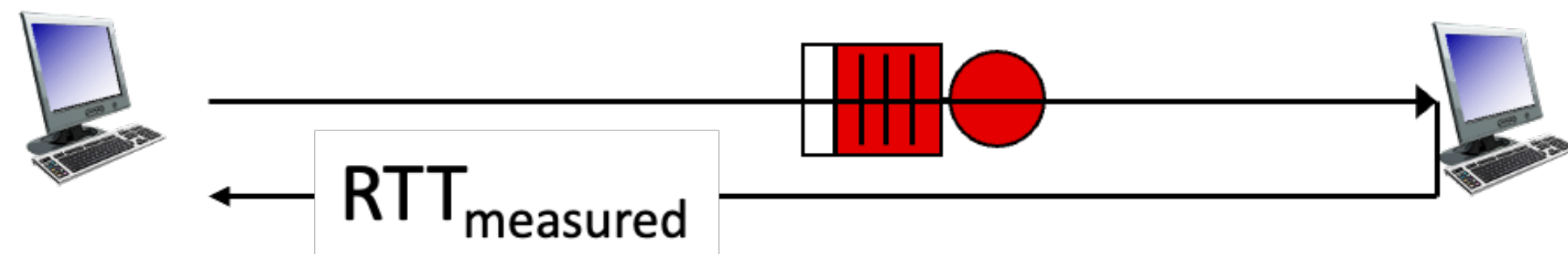
TCP & Congested bottleneck link

- TCP (classic, CUBIC) increases TCP's sending rate until packet loss occurs at some router's output → the bottleneck link
- Understanding congestion: useful to focus on congested bottleneck link



Delay-based TCP congestion control

- Keeping sender-to-receiver pipe “just full enough, but no fuller”: keep bottleneck link busy transmitting, but avoid high delays/buffering



$$\text{Measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{RTT_{measured}}$$

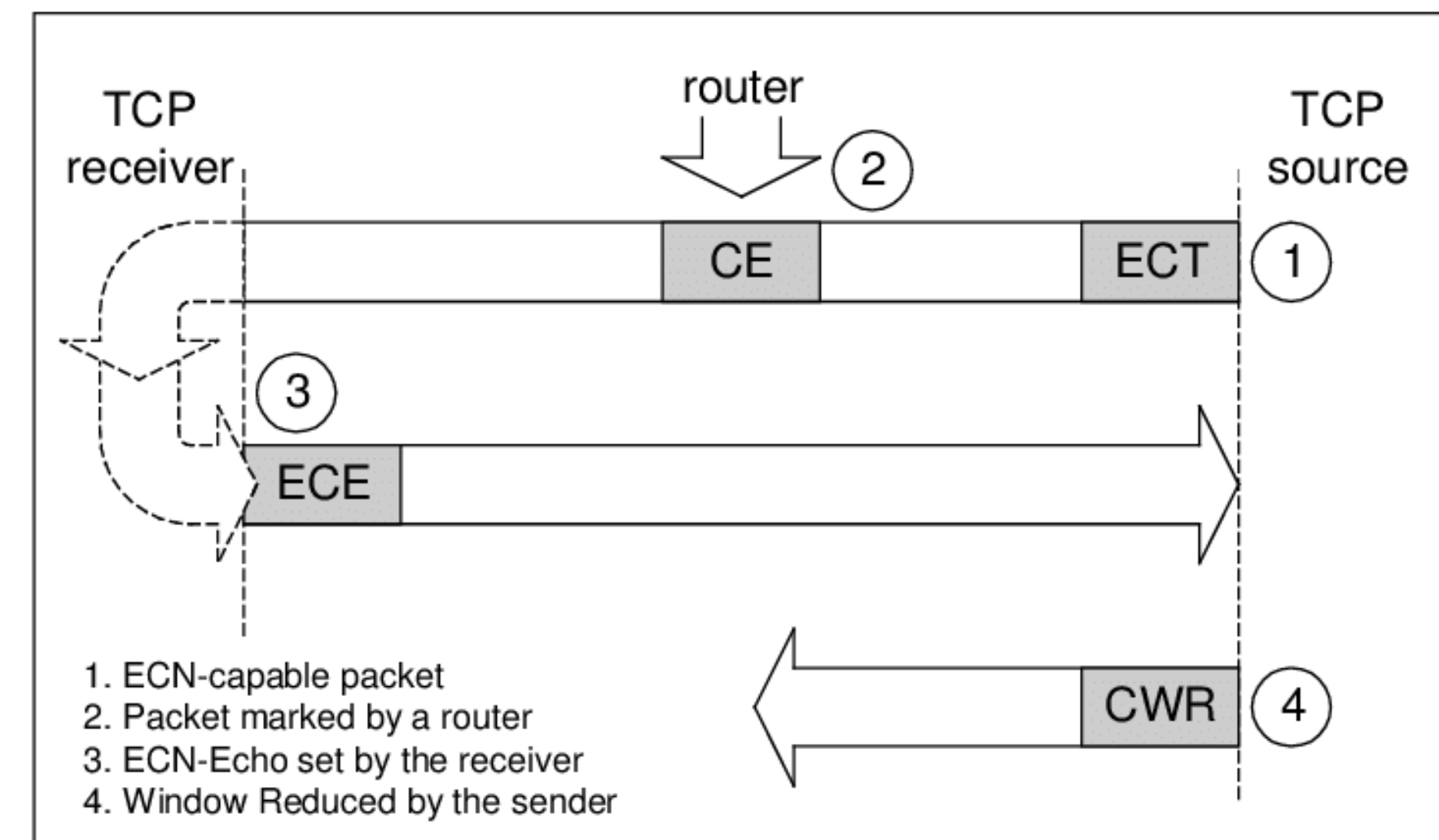
- Delay-based approach
 - RTT_{min} – minimum observed RTT (over a uncongested path)
 - Uncongested throughput with CWND is $cwnd/RTT_{min}$
 - Controlling CWND
 - If measured throughput is “very close” to uncongested throughput,
increase CWND **linearly** /* since path is not congested */
 - else If measured throughput is “far below” uncongested throughput,
decrease CWND **linearly** /* since path is congested */

Delay-based TCP congestion control

- Congestion control without inducing/forcing loss
- Maximizing throughput (“keeping the pipe just full...” while keeping delay low (“...but not fuller”)
- A number of delayed TCPs take a delay-based approach
 - BBR deployed on Google’s (internal) backbone network

Explicit congestion notification (ECN)

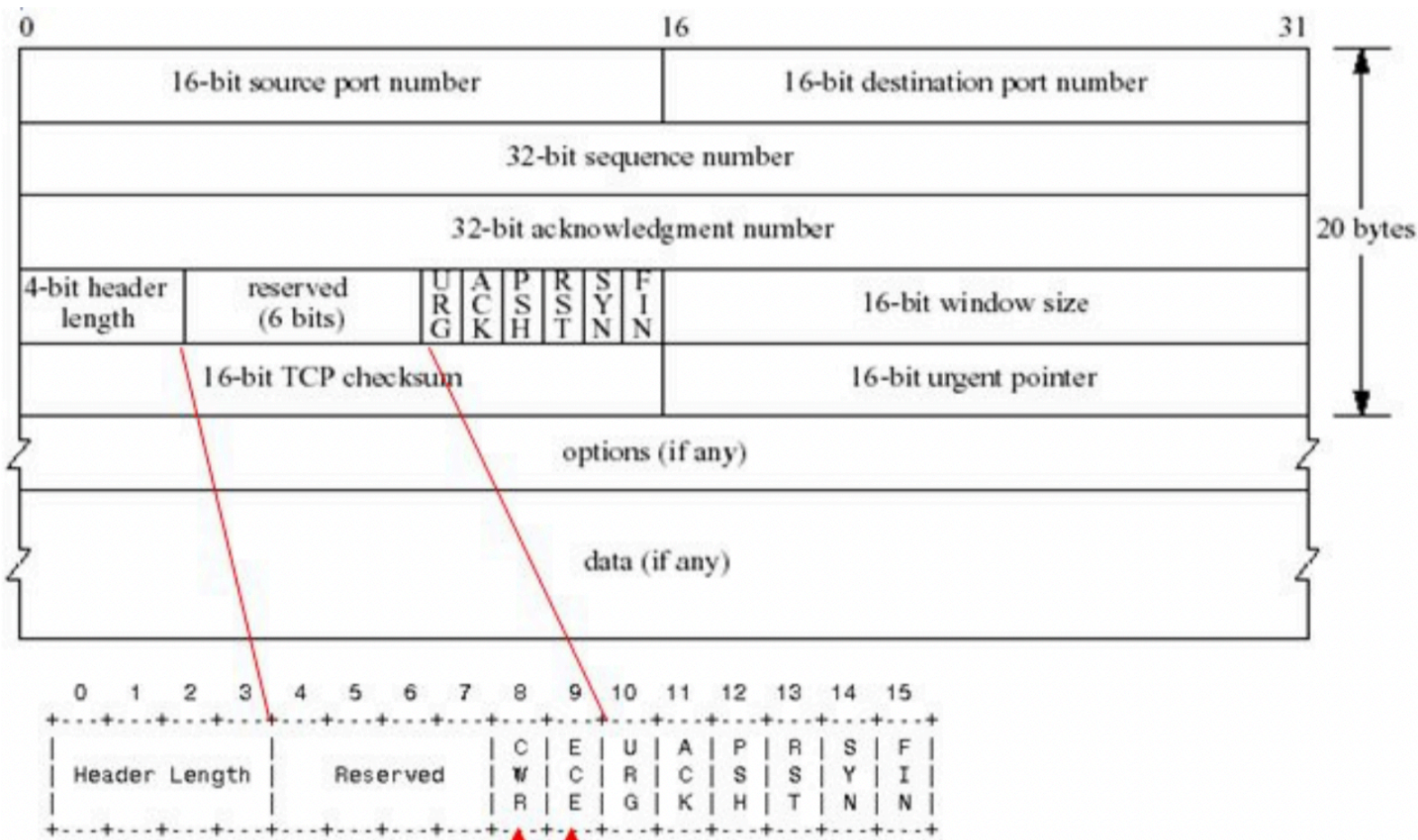
- TCP deployments often implement *network-assisted* congestion control
 - Two bits in IP header (ToS field) are marked by network routers to indicate congestion
 - Policy to determine marking can be chosen by network operator
- Congestion indication is carried to destination (**CE** in IP header)
- Destination sets **ECE** bit on ACK segment to notify sender of congestion (**ECE** in TCP header)
- Sender sets **CWR** bit to notify receiver of having processed ECE bit (**CWR** in TCP header)



Explicit congestion notification (ECN)

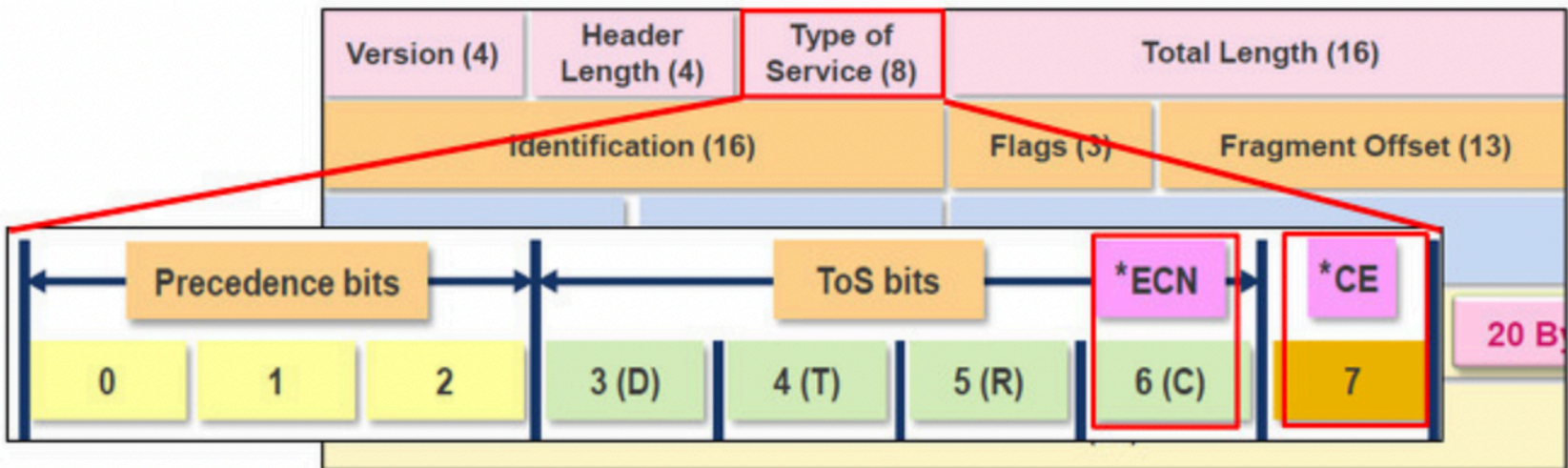
- Involve both IP (**ECN** bit in IP header) and TCP (**C**, **E** bit in TCP header)
- TCP header & IP header

TCP header



ECE flag - ECN-Echo flag
CWR flag - Congestion Window Reduced flag

IP header



ECN Field



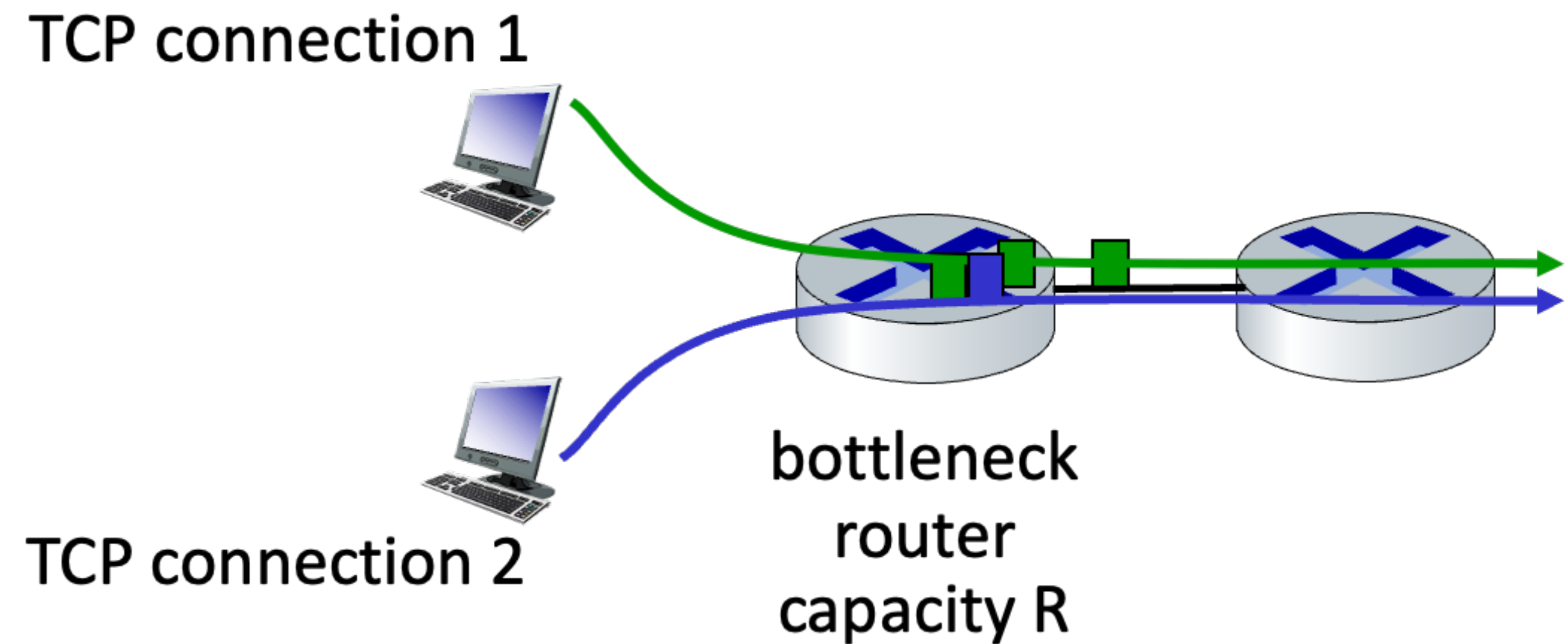
ECT: ECN Capable Transport
CE: Congestion Experienced

2 bits = 4 ECN Codepoints

ECT	CE	Names for the ECN bits
0	0	Not-ECT (Not ECN Capable Transport)
0	1	ECT(1) (ECN Capable Transport (1))
1	0	ECT(0) (ECN Capable Transport(0))
1	1	CE (Congestion Experienced)

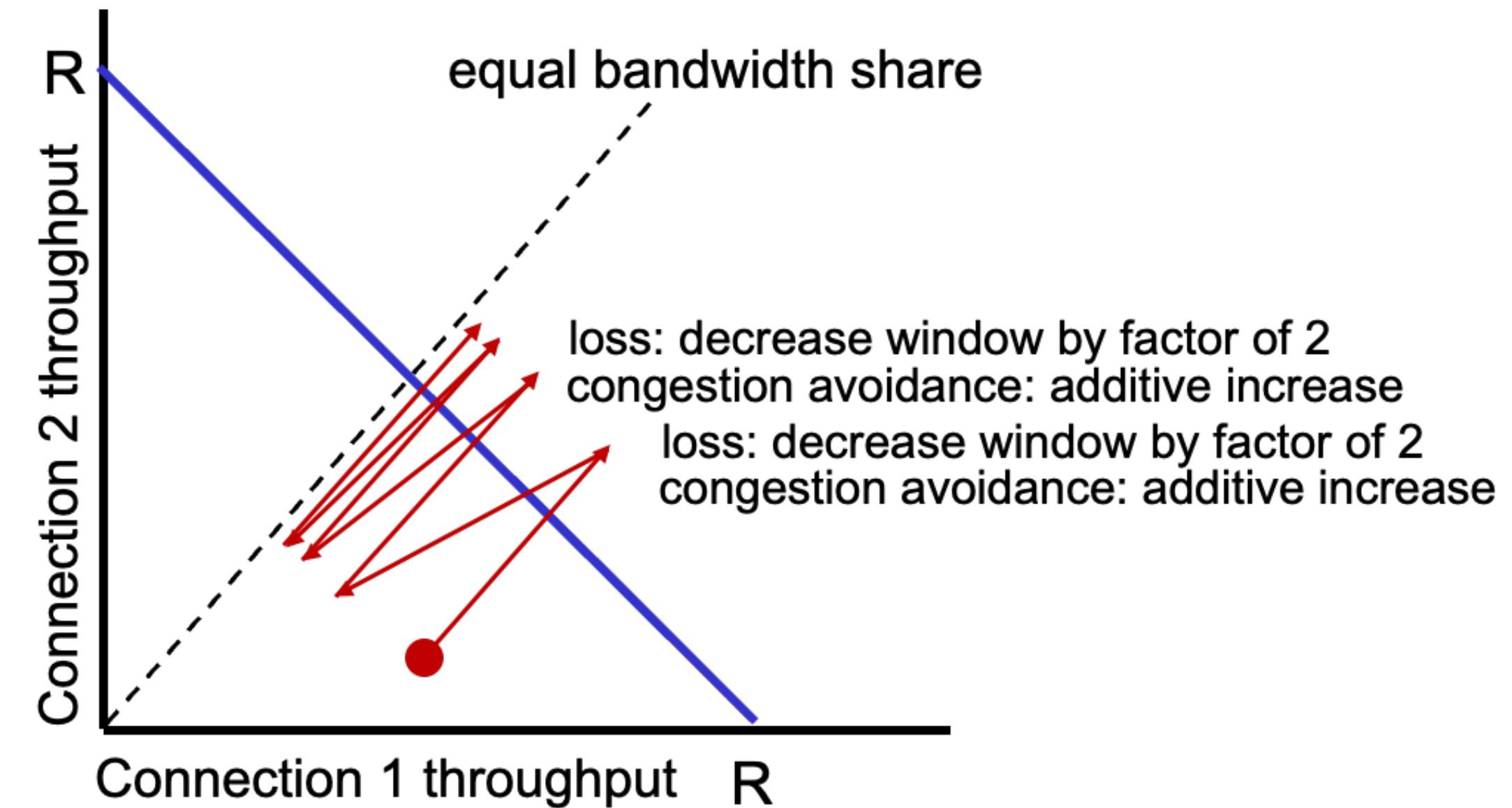
TCP fairness

- Fairness goal: if N TCP sessions share the same bottleneck link of bandwidth R , each should have average rate of $\frac{R}{N}$



Is TCP AIMD fair?

- e.g., Two competing TCP sessions
 - Additive increase gives a slope of 1, as throughput increases
 - Multiplicative decrease decreases throughput proportionally
- TCP is fair under ideal assumption of
 - Same RTT
 - Fixed number of sessions only in congestion avoidance



Fairness: must all network apps be “fair”?

- Fairness and UDP
 - Multimedia apps often do not use TCP
 - Do not want rate throttled by congestion control
 - Instead use UDP
 - Send audio/video at constant rate, tolerate packet loss
 - There is no “Internet police” policing use of congestion control
- Fairness and parallel TCP connections
 - Application can open multiple parallel connections between two hosts
 - Web browsers do this. e.g., link of rate R with 9 existing connections
 - New app asks for 1 TCP, gets a rate of $R/10$
 - New app asks for 11 TCPs, gets a rate of $R/2$

Questions?